

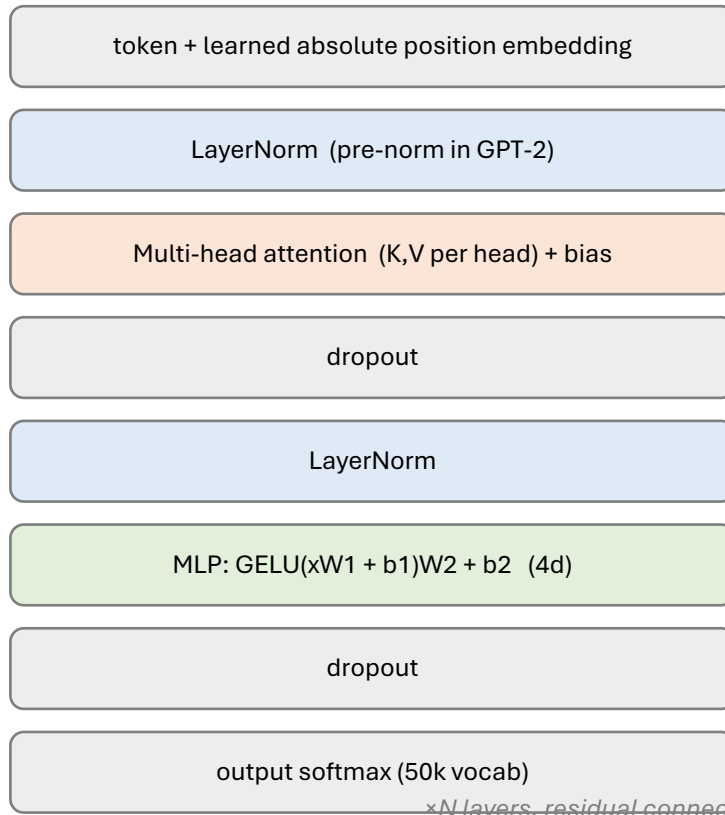
Deep Learning

Transformer architectures in 2026 — and looping

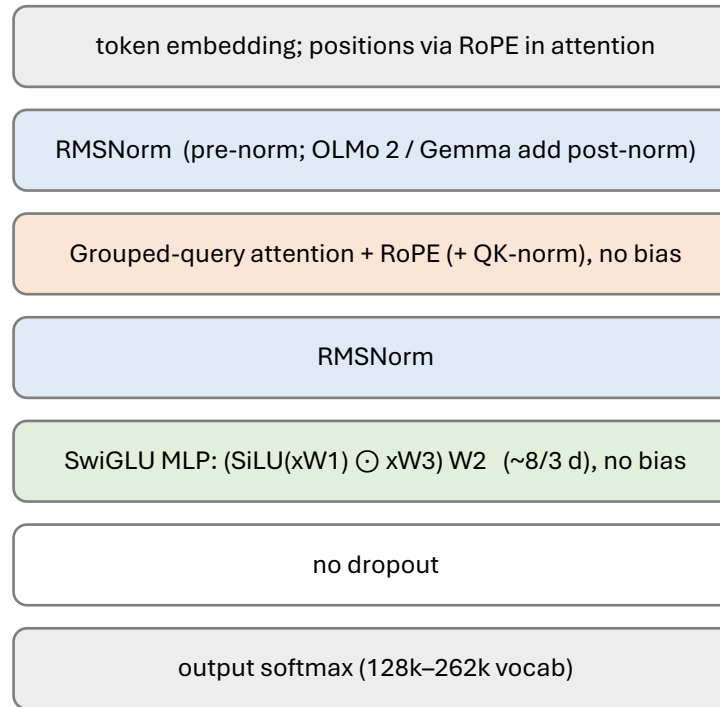
Partially based on S. Raschka's LLM Architecture Gallery (2026)

The block you learned vs. the block used today

GPT-2 block (2019)



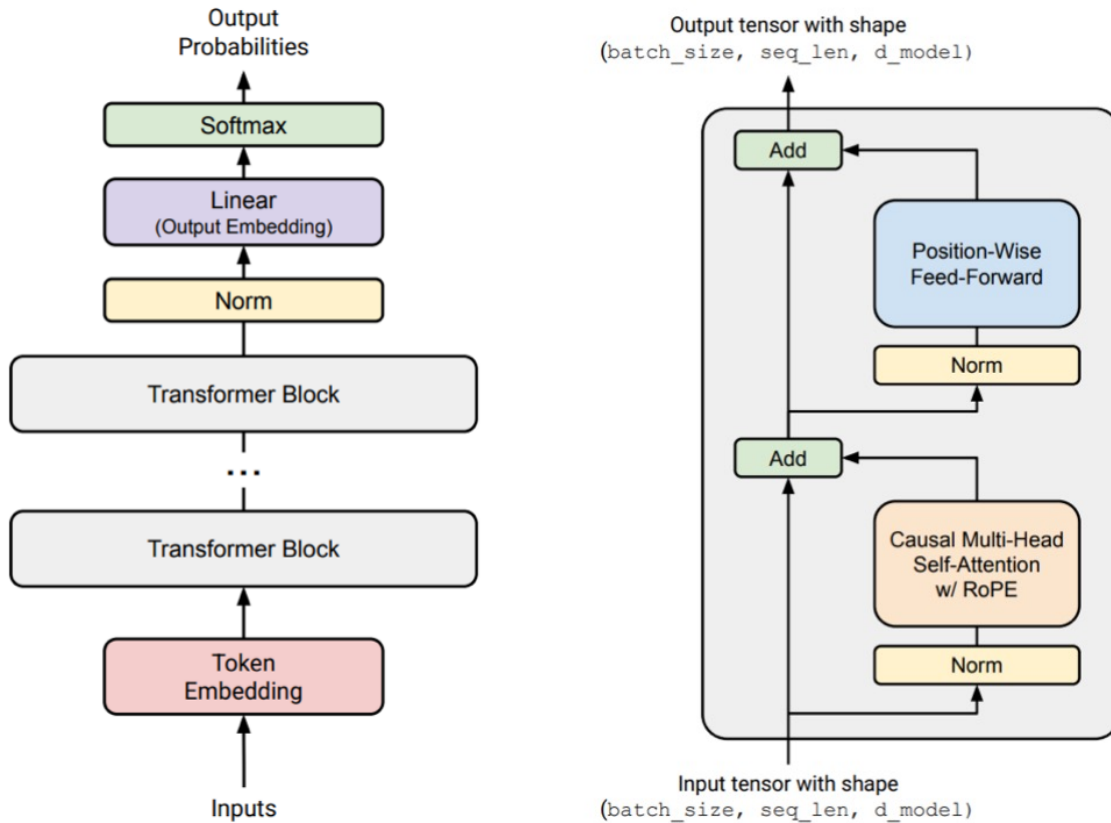
Llama-3 / Qwen3-style block (2024–25)



- Same skeleton: attention + MLP with residual connections
- Every part swapped for a cheaper or more stable variant; gone: dropout, biases, the learned position table
- Vocabulary 50k → 128k (Llama 3) – 262k (Gemma 3)

×N layers, residual connections around attention and MLP in both

What you implemented – simple, modern variant



Differences:

- LayerNorm is in front of the block
- Rotary position embeddings (RoPE)

FF layers use SwiGLU, not ReLU

- Linear layers (and layernorm) have no bias (constant) terms

Why did we pick these?

What should you pick?

Architecture variations..

Let's think about the core architecture piece

Aa Name	# Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)	2019	RMSNorm	Serial	☒	Relative	ReLU
GPT2	2019	LayerNorm	Serial	☒	Sine	GeLU
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	☒	Relative	GeGLU
mT5	2020	RMSNorm	Serial	☒	Relative	GeGLU
GPT3 (175B)	2020	LayerNorm	Serial	☒	Sine	GeLU
GPTJ	2021	LayerNorm	Parallel	☒	RoPE	GeLU
LaMDA	2021			☒	Relative	GeGLU
Gopher (280B)	2021	RMSNorm	Serial	☒	Relative	ReLU
GPT-NeoX	2022	LayerNorm	Parallel	☒	RoPE	GeLU
BLOOM (175B)	2022	LayerNorm	Serial	☒	Alibi	GeLU
OPT (175B)	2022	LayerNorm	Serial	☒	Absolute	ReLU
PaLM (540B)	2022	RMSNorm	Parallel	☒	RoPE	SwiGLU
Chinchilla	2022	RMSNorm	Serial	☒	Relative	ReLU
Mistral (7B)	2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA2 (70B)	2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA (65B)	2023	RMSNorm	Serial	☒	RoPE	SwiGLU
Qwen (14B)	2024	RMSNorm	Serial	☒	RoPE	SwiGLU
DeepSeek (67B)	2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Yi (34B)	2024	RMSNorm	Serial	☒	RoPE	SwiGLU

High level view:

- Low consensus
(except pre-norm)
- Trends toward 'LLaMA-like'
architectures

Why each change? (one line each)

- **RMSNorm** (Zhang & Sennrich, 2019): drop LayerNorm's mean-centering and bias — cheaper, same quality
- **Pre-norm**: normalize before each sublayer → stable deep training; OLMo 2 and Gemma 3 add a post-norm inside the residual as well
- **SwiGLU** (Shazeer, 2020): a gated MLP gives better loss per parameter than ReLU/GELU MLPs
- **RoPE** (Su et al., 2021): rotate q and k by position → only the relative offset enters the dot product; no learned table; extends to longer contexts by scaling (YaRN)
- **GQA** (Ainslie et al., 2023): share K,V across groups of heads → KV cache ~8× smaller in Llama-3-70B (64 query heads, 8 KV heads) at negligible loss
- **QK-norm** (Dehghani et al., ViT-22B, 2023): normalize q and k before the dot product → no attention-logit blow-ups; standard in OLMo 2, Gemma 3, Qwen3

RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}} \odot g$$

SwiGLU

$$\text{FFN}(x) = (\text{SiLU}(x W_1) \odot x W_3) W_2$$

RoPE: only $n-m$ matters

$$\langle R_m q, R_n k \rangle = \langle q, R_{n-m} k \rangle$$

QK-norm

$$\text{softmax}\left(\frac{\hat{q} \hat{k}^\top}{\sqrt{d}}\right) v, \quad \hat{q} = \text{RMSNorm}(q)$$

Pre-vs-post norm

The one thing everyone agrees on (in 2024)

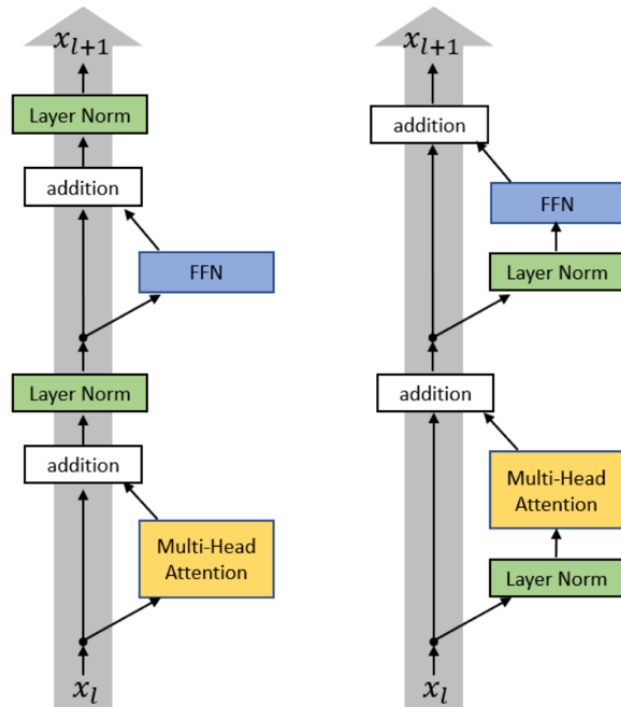


Figure from xiong2020

Post-LN Transformer	Pre-LN Transformer
$x_{l,i}^{post,1} = \text{MultiHeadAtt}(x_{l,i}^{post}, [x_{l,1}^{post}, \dots, x_{l,n}^{post}])$	$x_{l,i}^{pre,1} = \text{LayerNorm}(x_{l,i}^{pre})$
$x_{l,i}^{post,2} = x_{l,i}^{post} + x_{l,i}^{post,1}$	$x_{l,i}^{pre,2} = \text{MultiHeadAtt}(x_{l,i}^{pre,1}, [x_{l,1}^{pre,1}, \dots, x_{l,n}^{pre,1}])$
$x_{l,i}^{post,3} = \text{LayerNorm}(x_{l,i}^{post,2})$	$x_{l,i}^{pre,3} = x_{l,i}^{pre} + x_{l,i}^{pre,2}$
$x_{l,i}^{post,4} = \text{ReLU}(x_{l,i}^{post,3} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$	$x_{l,i}^{pre,4} = \text{LayerNorm}(x_{l,i}^{pre,3})$
$x_{l,i}^{post,5} = x_{l,i}^{post,3} + x_{l,i}^{post,4}$	$x_{l,i}^{pre,5} = \text{ReLU}(x_{l,i}^{pre,4} W^{1,l} + b^{1,l}) W^{2,l} + b^{2,l}$
$x_{l+1,i}^{post} = \text{LayerNorm}(x_{l,i}^{post,5})$	$x_{l+1,i}^{pre} = x_{l,i}^{pre,5} + x_{l,i}^{pre,3}$
Final LayerNorm: $x_{Final,i}^{pre} \leftarrow \text{LayerNorm}(x_{L+1,i}^{pre})$	

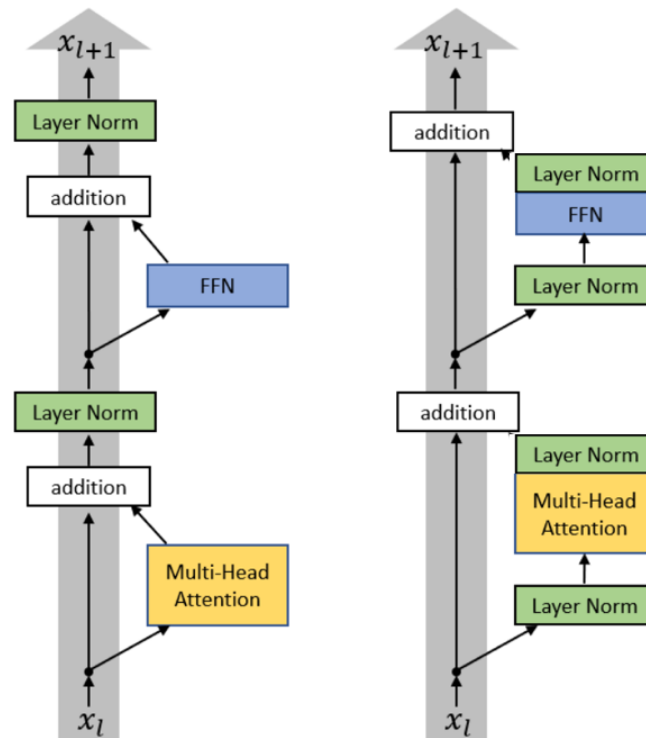
Set up LayerNorm so that it doesn't affect the main residual signal path (on the left)

Almost all modern LMs use pre-norm (but BERT was post-norm)

(One somewhat funny exception – OPT350M. I don't know why this is post-norm)

New things – ‘double’ norm.

If putting LayerNorms in residual streams is bad.. Why not post-norm outside the stream?



Recent models: Grok, Gemma 2. Olmo 2 only does non-residual post norm

LayerNorm vs RMSNorm

Original transformer: **LayerNorm** – normalizes the mean and variance across d_{model}

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

Many modern LMs: RMSNorm – does not subtract mean or add a bias term

$$y = \frac{x}{\sqrt{\|x\|_2^2 + \epsilon}} * \gamma$$

Notable models:

GPT3/2/1, OPT, GPT-J, BLOOM

Notable models:

LLaMA-family, PaLM, Chinchilla, T5

Why RMSNorm?

Modern explanation – it's faster (and just as good).

- Fewer operations (no mean calculation) Fewer parameters (no bias term to store)

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

Does this explanation make sense?

Operator class	% flop
Δ Tensor contraction	99.80
$\square$ Stat. normalization	0.17
$\circ$ Element-wise	0.03

Matrix multiplies are the vast majority of FLOPs (and memory)

More generally: dropping bias terms

Most modern transformers don't have bias terms.

Original Transformer:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Most implementations (if they're not gated):

$$\text{FFN}(x) = \sigma(xW_1)W_2$$

Reasons: memory (similar to RMSnorm) and optimization stability

Gated activations (*GLU)

GLUs modify the ‘first part’ of a FF layer

$$FF(x) = \max(0, xW_1) W_2$$

Instead of a linear + ReLU, augment the above with an (entrywise) linear term

$$\max(0, xW_1) \rightarrow \max(0, xW_1) \otimes (xV)$$

This gives the gated variant (ReGLU) – note that we have an extra parameter (V)

$$FF_{\text{ReGLU}}(x) = (\max(0, xW_1) \otimes xV) W_2$$

Do gated linear units work?

Yes, fairly consistently so.

	Score Average	CoLA MCC	SST-2 Acc
FFN _{ReLU}	83.80	51.32	94.04
FFN _{GELU}	83.86	53.48	94.04
FFN _{Swish}	83.60	49.79	93.69
FFN _{GLU}	84.20	49.16	94.27
FFN _{GEGLU}	84.12	53.65	93.92
FFN _{Bilinear}	83.79	51.02	94.38
FFN _{SwiGLU}	84.36	51.59	93.92
FFN _{ReGLU}	84.67	56.16	94.38
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Shazeer 2020

Summary: architectures

Pre-vs-post norm: Everyone does pre-norm (except

- OPT350M), likely with good reason.

Layer vs RMSnorm:

RMSnorm has clear compute wins,

- sometimes even performance

Gating:

- GLUs seem generally better, though differences are small

Serial vs parallel layers:

- No extremely serious ablations, but has a comp win.

As Name	#	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer		2017	LayerNorm	Serial	☐	Sine	ReLU
GPT		2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)		2019	RMSNorm	Serial	☒	Relative	ReLU
GPT2		2019	LayerNorm	Serial	☒	Absolute	GeLU
T5 (XXL 11B) v1.1		2020	RMSNorm	Serial	☒	Relative	GeGLU
mT5		2020	RMSNorm	Serial	☒	Relative	GeGLU
GPT3 (175B)		2020	LayerNorm	Serial	☒	Absolute	GeLU
GPTJ		2021	LayerNorm	Parallel	☒	RoPE	GeLU
LaMDA		2021			☒	Relative	GeGLU
Anthropic LM (not claude)		2021			☒		
Gopher (280B)		2021	RMSNorm	Serial	☒	Relative	ReLU
GPT-NeoX		2022	LayerNorm	Parallel	☒	RoPE	GeLU
BLOOM (175B)		2022	LayerNorm	Serial	☒	Alibi	GeLU
OPT (175B)		2022	LayerNorm	Serial	☒	Absolute	ReLU
PaLM (540B)		2022	RMSNorm	Parallel	☒	RoPE	SwiGLU
Chinchilla		2022	RMSNorm	Serial	☒	Relative	ReLU
Mistral (7B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA2 (70B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
LLaMA (65B)		2023	RMSNorm	Serial	☒	RoPE	SwiGLU
GPT4		2023			☐		
BaiChuan 2		2023	RMSNorm	Serial	☒	Alibi	SwiGLU
Olmo 2		2024	RMSNorm	Serial	☐	RoPE	SwiGLU
Gemma 2 (27B)		2024	RMSNorm	Serial	☒	RoPE	GeGLU
Nemotron-4 (340B)		2024	LayerNorm	Serial	☒	RoPE	SqReLU
Qwen 2 (72b) - same for 2.5		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Falcon 2 11B		2024	LayerNorm	Parallel	☒	RoPE	GeLU
Phi3 (small) - same for phi4		2024	RMSNorm	Serial	☒	RoPE	GeGLU
Llama 3 (70B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Reka Flash		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Command R+		2024	LayerNorm	Parallel	☒	RoPE	SwiGLU
OLMo		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Qwen (14B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
DeepSeek (67B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Yi (34B)		2024	RMSNorm	Serial	☒	RoPE	SwiGLU
Mixtral of Experts		2024			☐		
Command A		2024	LayerNorm	Parallel	☒	Hybrid (RoPE+NoPE)	SwiGLU
Gemma 3		2025	RMSNorm	Serial	☐	RoPE	GeGLU
SmoLM2 (1.7B)		2025	RMSNorm	Serial	☒	RoPE	SwiGLU

RoPE: rotary position embeddings

High level thought process: a *relative* position embedding should be some $f(x, i)$ s.t.

$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j)$$

That is, the attention function *only* gets to depend on the relative position (i-j). How do existing embeddings not fulfill this goal?

- **Sine:** Has various cross-terms that are not relative

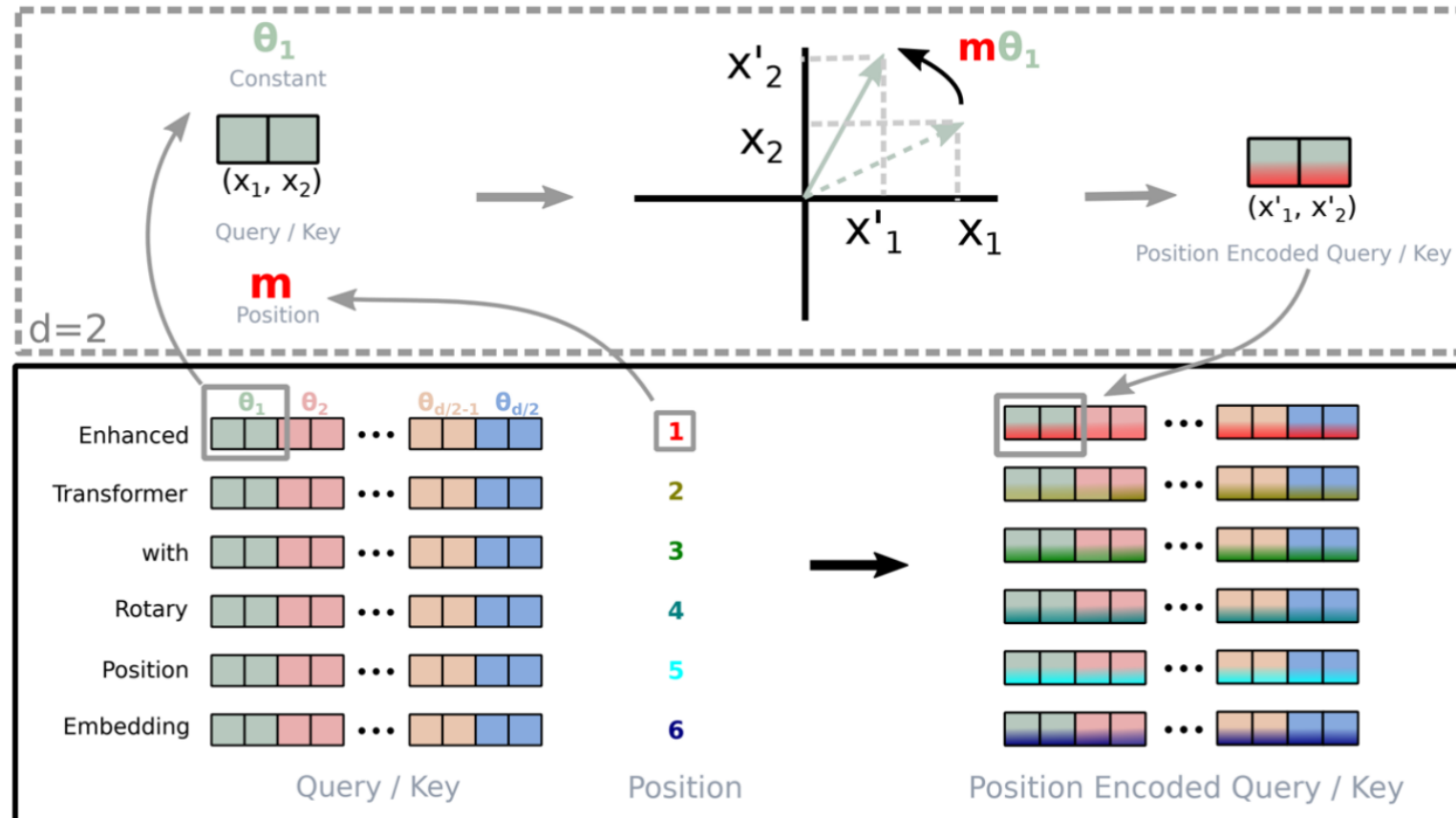
$$\langle \text{Embed}(x, i), \text{Embed}(y, i) \rangle = \langle v_x, v_y \rangle + \langle PE_i, v_y \rangle \dots$$

- **Absolute:** obviously not relative

- **Relative embeddings:** $e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$ is not an inner product

RoPE: rotary position embeddings

There are many rotations, which one do you pick?



[Su et al 2021]

Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)

Surprising (?) consensus hyperparameter 1

Feedforward – model dimension ratio.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

There are two dimensions that are relevant – the feedforward dim (d_{ff}) and model dim (d_{model}). What should their relationship be?

$$\mathbf{d_{ff} = 4 d_{model}}$$

This is *almost always* true. There's just a few exceptions.

Surprising (?) consensus hyperparameter 2

Head-dim*num-heads to model-dim ratio. As a reminder, slide from 224n.

Multi-head self-attention is computationally efficient

- Even though we compute h many attention heads, it's not really more costly.
 - We compute $XQ \in \mathbb{R}^{n \times d}$, and then reshape to $\mathbb{R}^{n \times h \times d/h}$. (Likewise for XK, XV .)
 - Then we transpose to $\mathbb{R}^{h \times n \times d/h}$; now the head axis is like a batch axis.
 - Almost everything else is identical, and the **matrices are the same sizes**.

This doesn't have to be true: we can have head-dimensions $>$ model-dim / num-heads.

But most models do follow this guideline

What are typical vocabulary sizes?

Monolingual models – 30-50k vocab

Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000

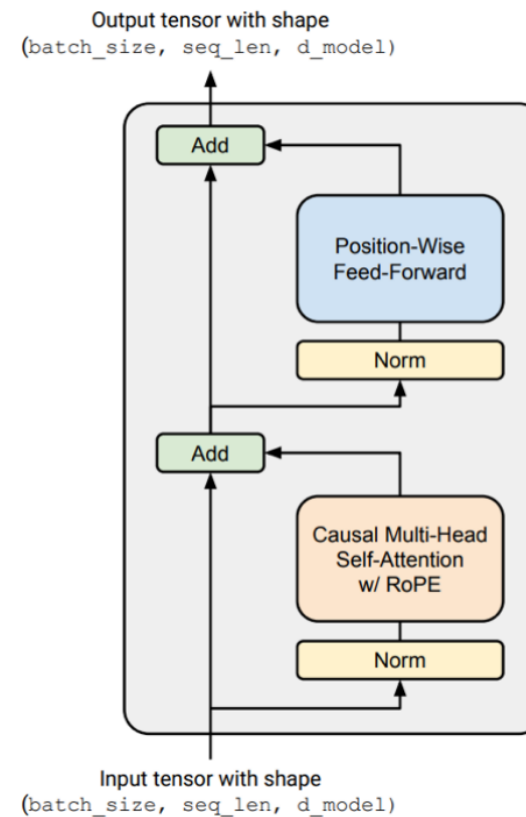
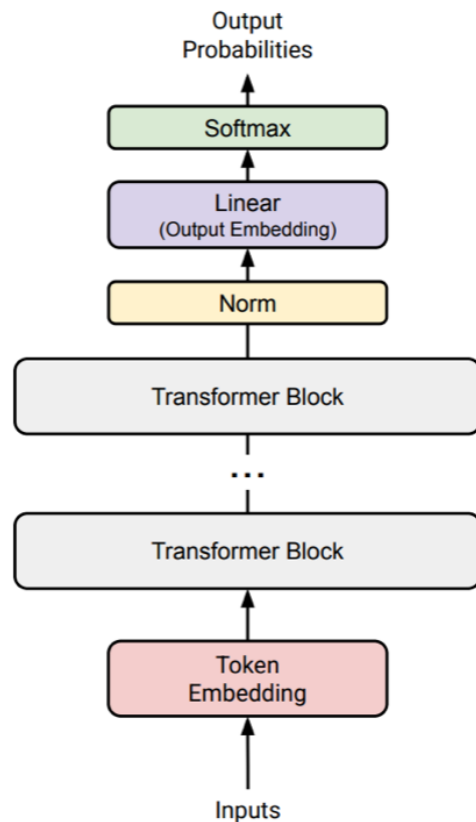
Multilingual / production systems 100-250k

Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
Command A	255000
DeepSeek	100000
Qwen 15B	152064
Yi	64000

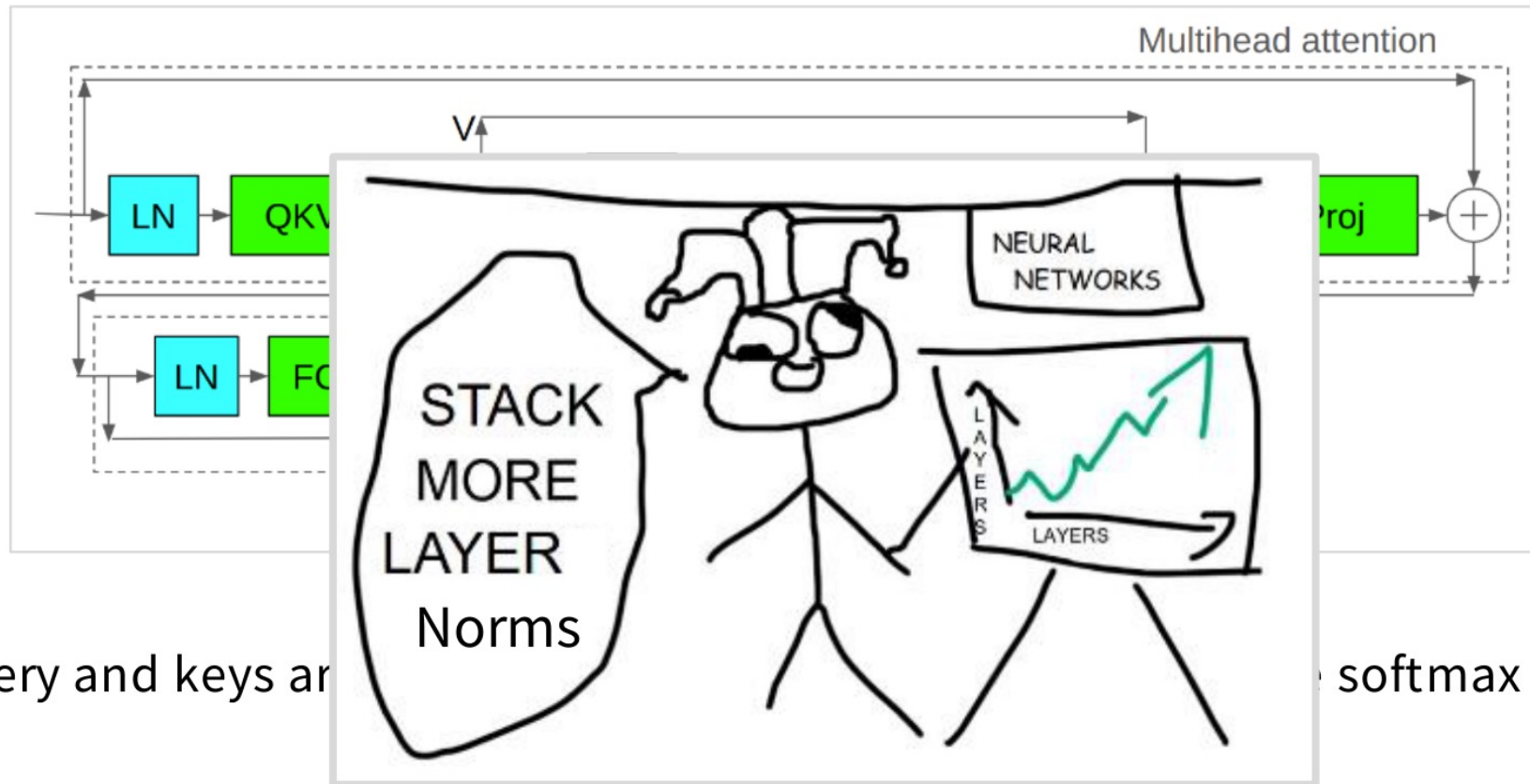
Monolingual vocabs don't need to be huge, but multilingual ones do

Where do the issues arise? Beware of softmaxes!

Softmaxes – can be ill-behaved due to exponentials / division by zero



Attention softmax stability – the ‘QK norm’



The query and keys are

softmax operation.

Other examples: DCLM, OLMo2, Gemma 2

Originally from vision and multimodal models [Dehgani 2023, Idefcs, Chameleon]

Logit soft-capping.

Soft-capping the logits to some maximum value via Tanh

Logit soft-capping. We cap logits ([Bello et al., 2016](#)) in each attention layer and the final layer such that the value of the logits stays between $-\text{soft_cap}$ and $+\text{soft_cap}$. More specifically, we cap the logits with the following function:

$$\text{logits} \leftarrow \text{soft_cap} * \tanh(\text{logits}/\text{soft_cap}).$$

We set the `soft_cap` parameter to 50.0 for the self-attention layers and to 30.0 for the final layer.

Prevents logits from blowing up, but also might have perf issues?

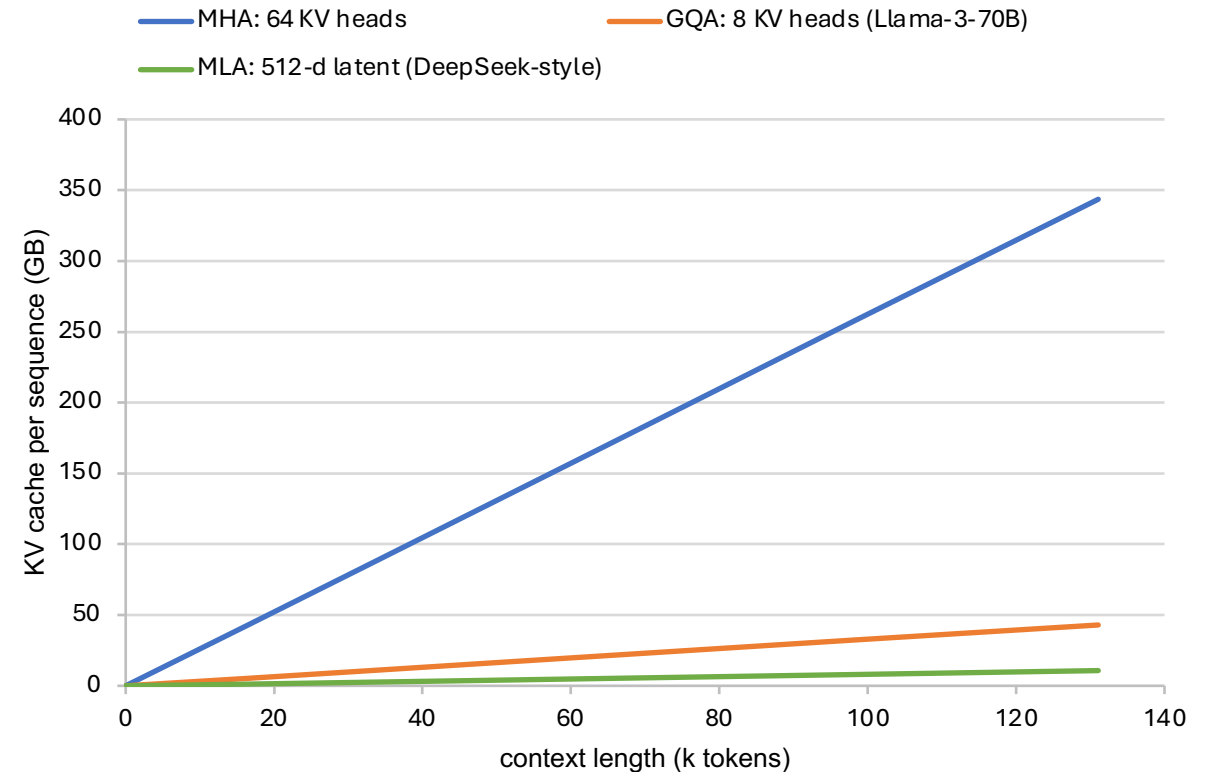
Table 4: Models perplexity with confidence interval ± 0.1 at 95% level.

bf16 baseline	<i>soft_cap</i>	<i>QKV_norm</i>	<i>QK_norm_cap</i>	<i>QK_norm</i>	<i>QK_FC_norm</i>
11.19	11.24	10.85	11.00	10.84	10.87

The KV cache is the inference bottleneck

KV cache per token per layer = $2 \times n_{kv} \times d_{head} \times \text{bytes per element}$

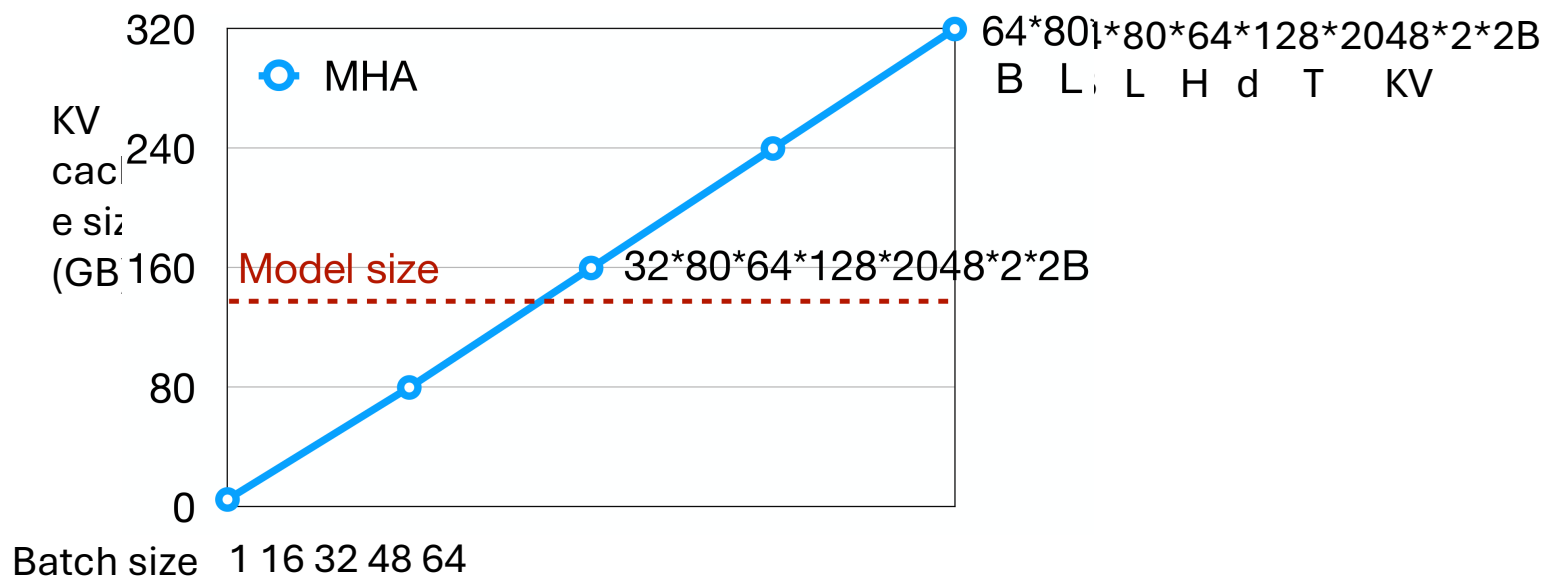
- Every decoded token re-reads the whole cache: memory and bandwidth, not FLOPs, bound long-context decoding
- Llama-3-70B-like, 128k tokens, BF16: **344 GB** with 64 KV heads → **43 GB** with GQA (8 KV heads) → **11 GB** with a 512-d MLA latent
- Long contexts (128k–1M) make the cache dominate → GQA → MLA → sliding windows → hybrids with linear layers



KV Cache Optimizations

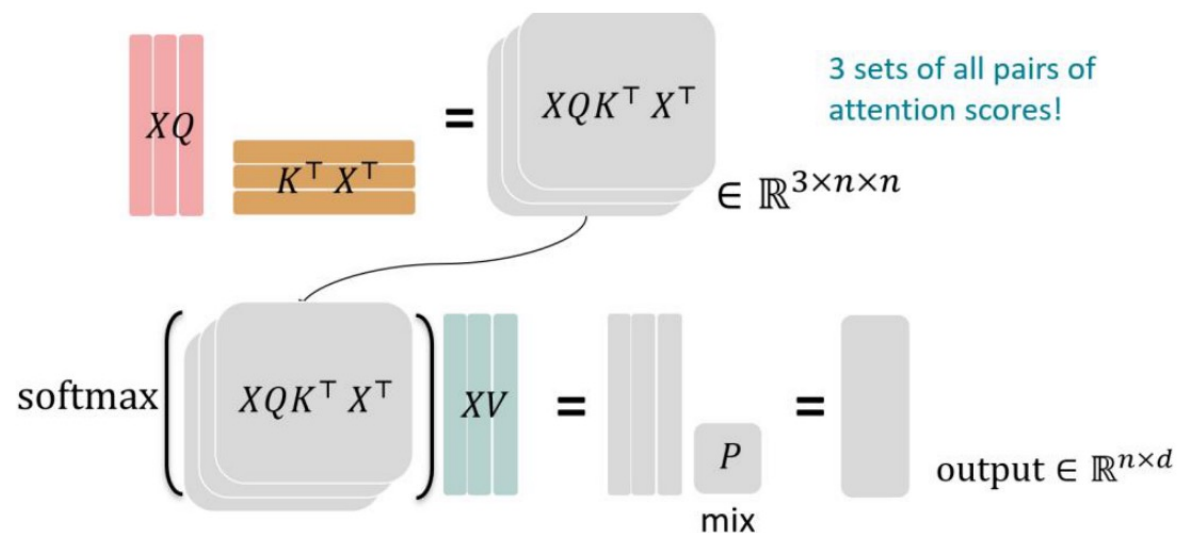
The KV cache could be large with long context

- Now, we calculate the KV cache size under $N = 2048$ different batch sizes
- The KV cache size goes quickly larger than the model weights



GQA/MQA – Reducing attention head cost

Let's think about the compute involved for attention



Total arithmetic operations (bnd^2) , **total memory accesses** $(bnd + bhn^2 + d^2)$

Arithmetic intensity is high $O\left(\left(\frac{1}{k} + \frac{1}{bn}\right)^{-1}\right)$ - we can keep our GPUs running

GQA/MQA – Reducing attention head cost

What's the incremental arithmetic intensity?

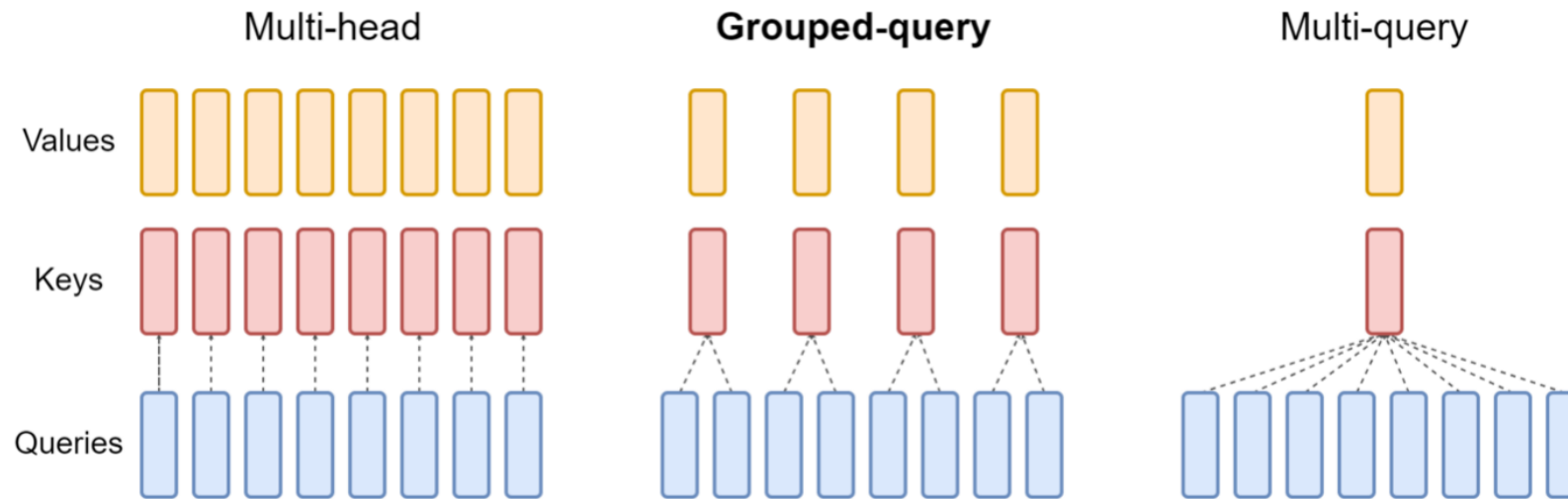
Total arithmetic operations (bnd^2) , **total memory accesses** $(bn^2d + nd^2)$ $\times$ $\frac{1}{b}$ projection

Arithmetic intensity is not good $O\left(\left(\frac{n}{d} + \frac{1}{b}\right)^{-1}\right)$ - need large batches + short seq length
(n) or big model dimensions (d)

Is there some way around this? The n/d term is difficult to reduce.

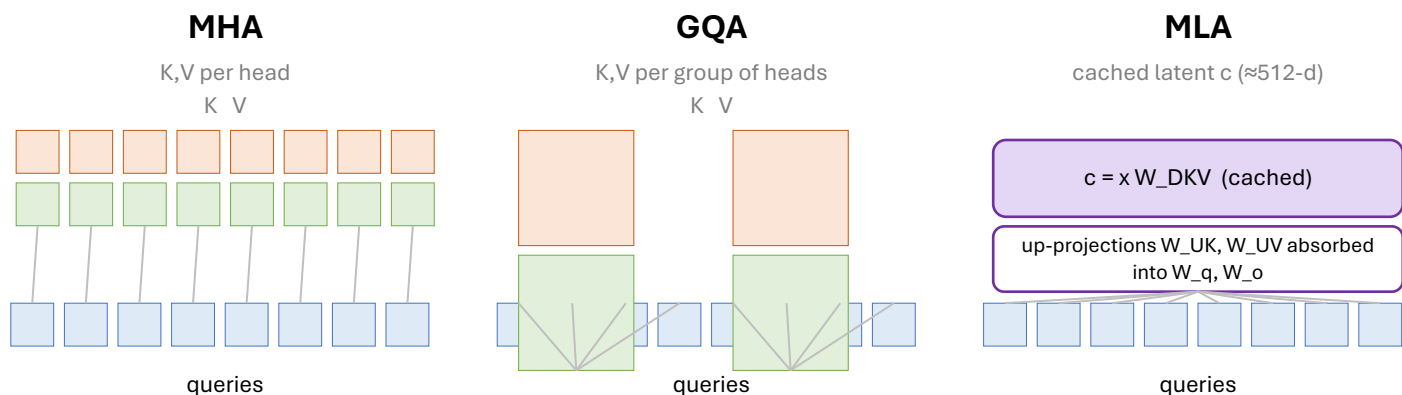
Recent extension – GQA

Don't go all the way to one dimension of KV – have fewer dims



Simple knob to control expressiveness (key-query ratio) and inference efficiency

MHA → GQA → MLA

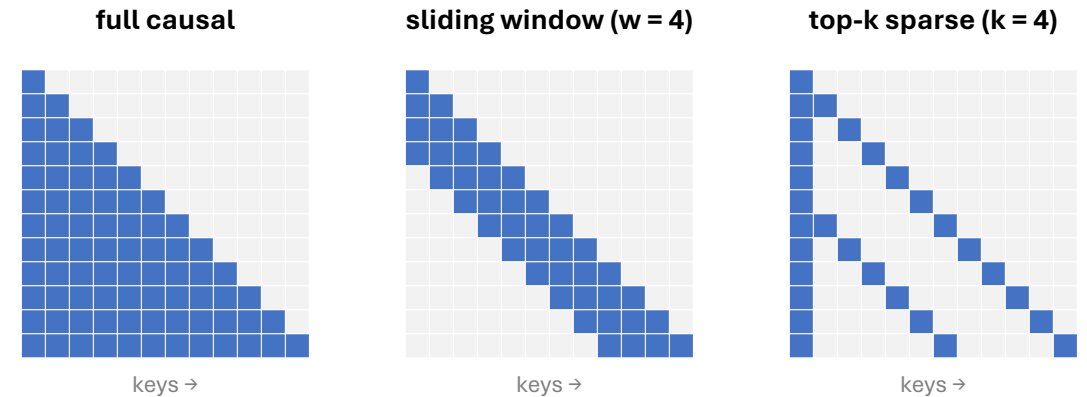


- **MQA** (Shazeer, 2019): one shared K,V for all heads — the extreme; **GQA** (2023) interpolates between MQA and MHA
- **MLA** (DeepSeek-V2, 2024): cache a ~ 512 -d latent c per token instead of K,V; the up-projections W_{UK}, W_{UV} are absorbed into W_q and W_o ; a small decoupled RoPE key carries position
- DeepSeek-V2 reports 93.3% less KV cache than DeepSeek-67B and, in its ablations, better quality than GQA at that cache size
- Adopted by DeepSeek V3/R1, Kimi K2, Mistral Large 3, GLM-5 — per the gallery, MHA → GQA → MLA is the dominant shift in large models

cache per token per layer: MHA $2 \cdot 8 \cdot d_h$ | GQA $2 \cdot 2 \cdot d_h$ | MLA ≈ 512 (+64 RoPE key)

Sparse and local attention

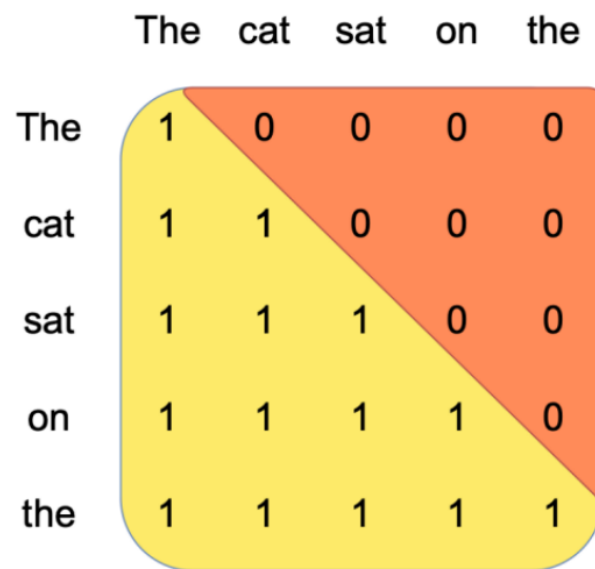
- **Sliding window** (width w): each layer keeps only the last w keys $\rightarrow$ constant KV per layer; mixed with a few global layers (table)
- **DeepSeek Sparse Attention** (V3.2, Dec 2025; also GLM-5, Feb 2026): a cheap 'lightning indexer' scores past tokens and each query attends only to its top-2048 keys $\rightarrow$ near-linear cost at long context
- **Attention sinks** (gpt-oss): a learned per-head bias logit in the softmax denominator absorbs attention mass — from StreamingLLM (Xiao et al., 2023)
- Local + global mixes go back to Longformer / BigBird (2020); what is new is the scale and the ratios



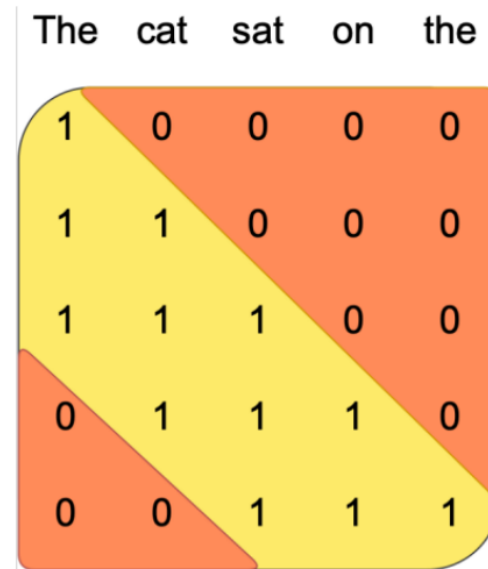
Model (date)	local : global	window
Gemma 3 (Mar 2025)	5 : 1	1024
gpt-oss (Aug 2025)	1 : 1 (alternating)	128
OLMo 3 (Nov 2025)	3 : 1	—
Gemma 4 (Apr 2026)	5 : 1	—

Sliding window attention

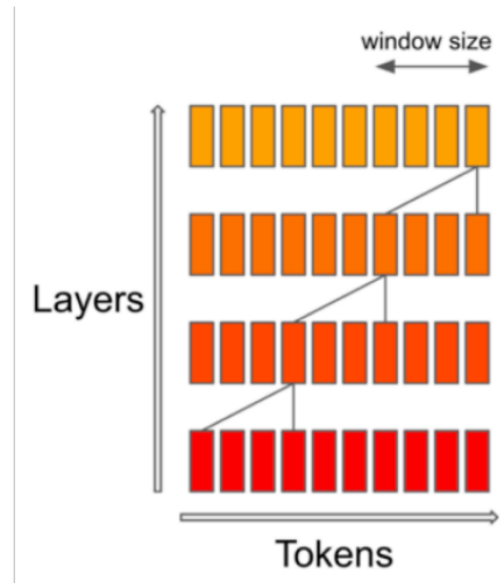
Another variation on this idea – sliding window attention



Vanilla Attention



Sliding Window Attention

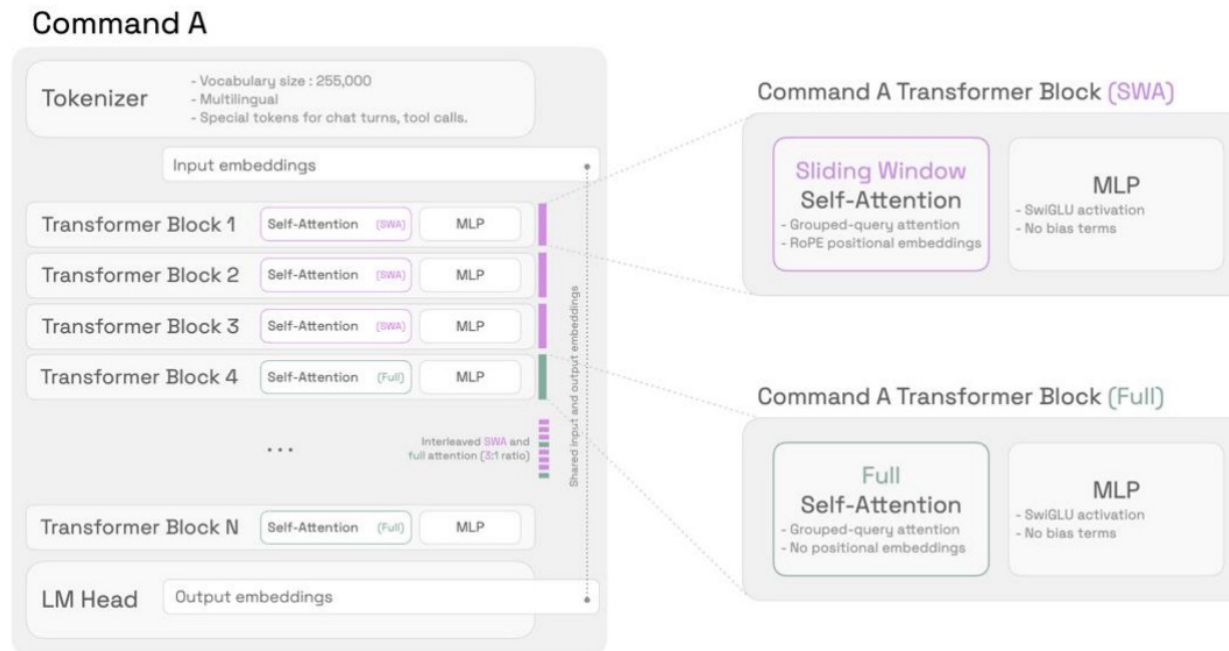


Effective Context Length

Just use the main part of the strided pattern – let depth extend effective context (Mistral)

Current standard trick – interleave ‘full’ and ‘LR’ attention

From Cohere Command A – Every 4th layer is a full attention



Long-range info via NoPE, short-range info via RoPE + SWA.

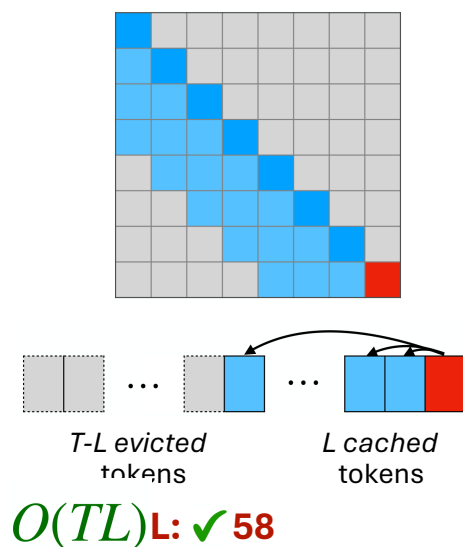
Other models – LLaMA 4, Gemma does SWA+Full RoPE.

The Limits of Window Attention

- A natural approach — window attention: caching only the most recent Key-Value states.
- Drawback: model collapses when the text length surpasses the cache size, when the initial token

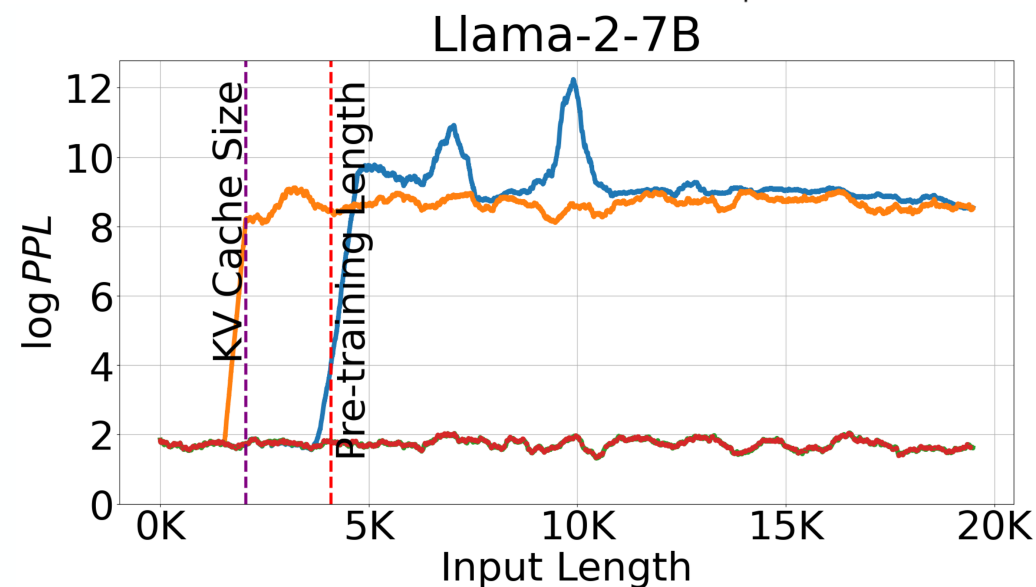
is evicted.

(b) Window Attention



Breaks when initial tokens are evicted.

— Dense Attention — Window Attention — Sliding Window w/ Re-computation — StreamingLLM



The “Attention Sink” Phenomenon

- We observed the phenomenon in the SpAtten project in 2021, but wan’t able to explain it until 2023:

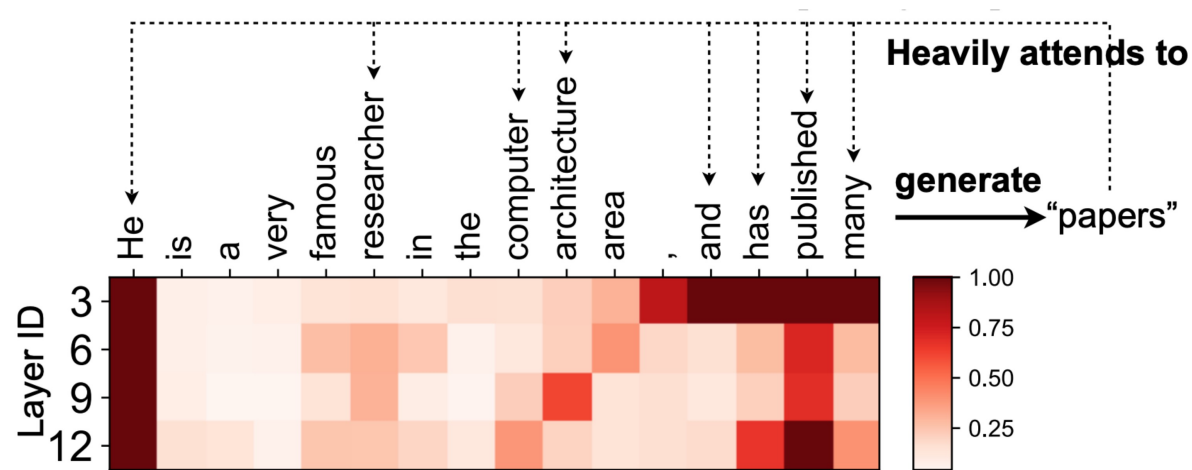


Fig. 23. Cumulative importance scores in GPT-2. Unimportant tokens are pruned on the fly. Important tokens are heavily attended.

GPT-2 for Language Modeling

Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English

~~Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English~~

~~Du Fu was a great poet of the Tang dynasty. Recently a variety of styles have been used in efforts to translate the work of Du Fu into English~~

‘English’ is the generated token.

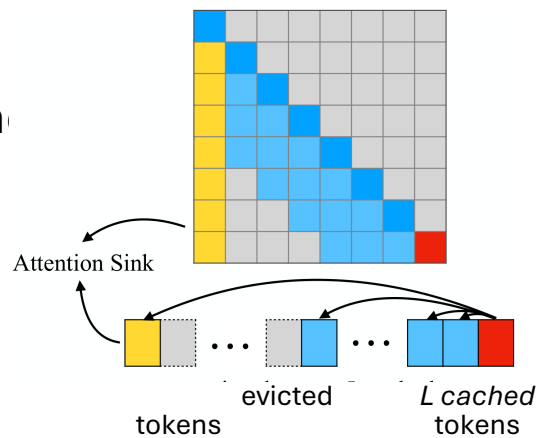
StreamingLLM: Using Attention Sinks for Infinite Streams

- **Objective: Enable LLMs trained with a finite attention window to handle infinite text lengths**

without additional training.

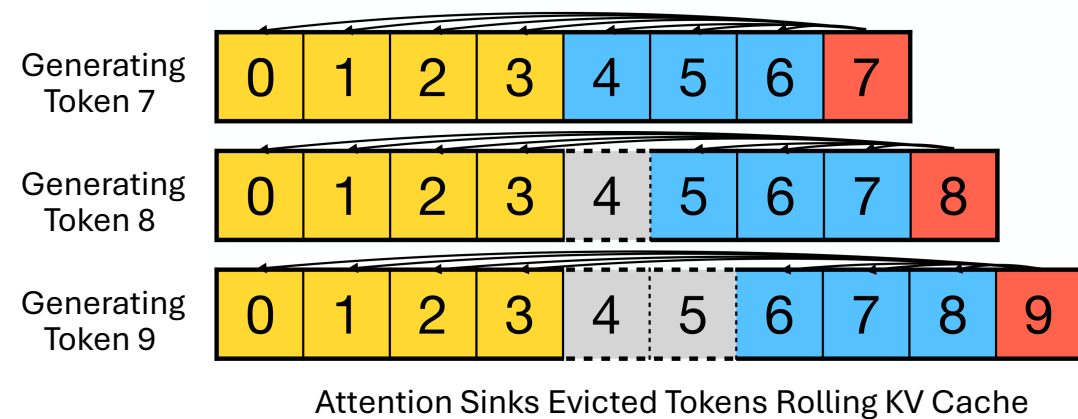
- **Key Idea: preserve the KV of attention sink tokens, along with the sliding window's KV to**

stabilize the model



$O(TL)$ PPL: 5.40

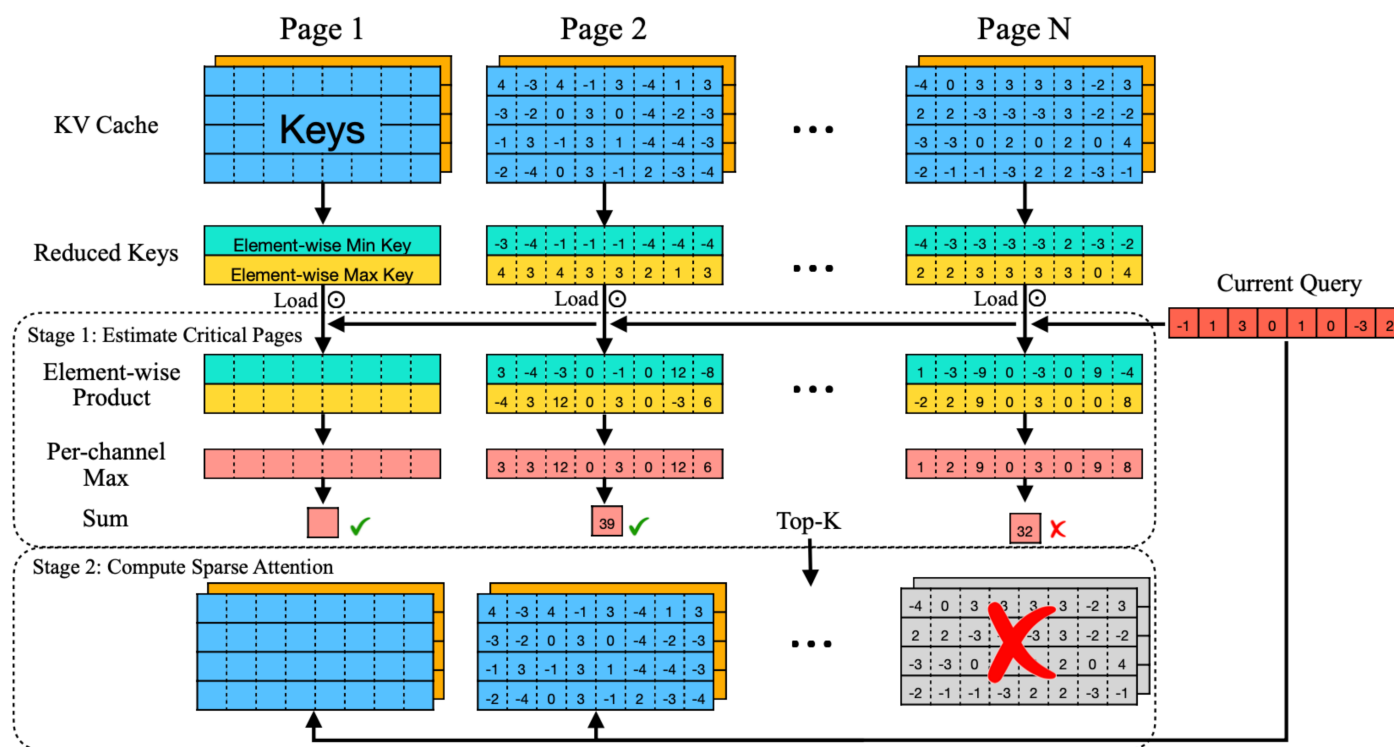
Can perform efficient and stable language modeling on long texts.



Quest: Using Query-aware Sparsity in Attention

- **Key Idea: preserve all KV cache, and significantly accelerate inference by reducing the memory**

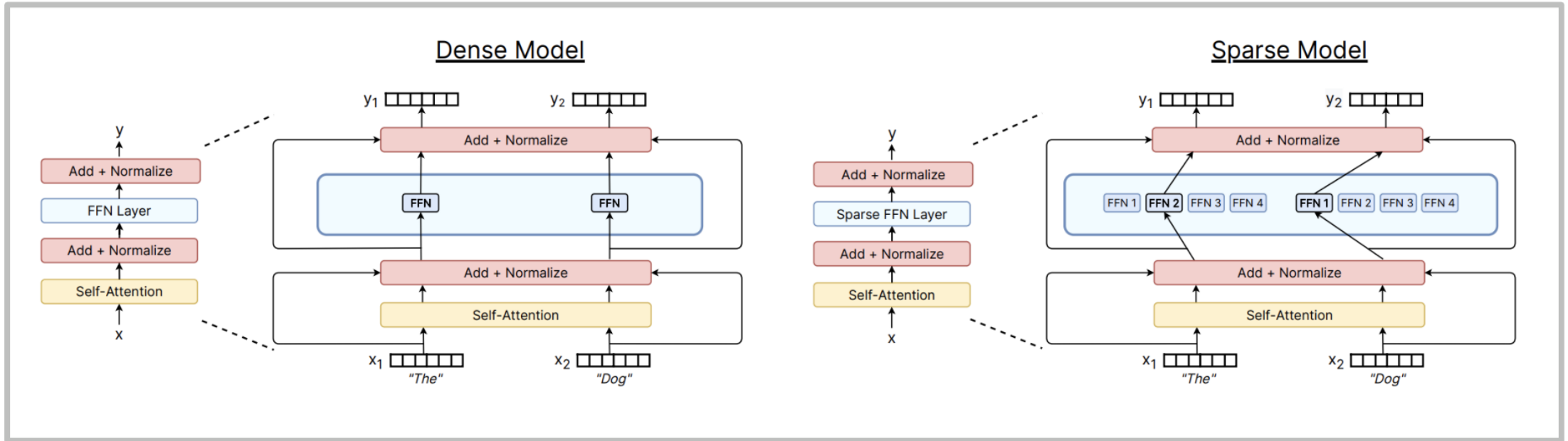
movement from



Mixture of experts

more parameters, same FLOPs — Stanford CS336 Lecture 4

What's a MoE?



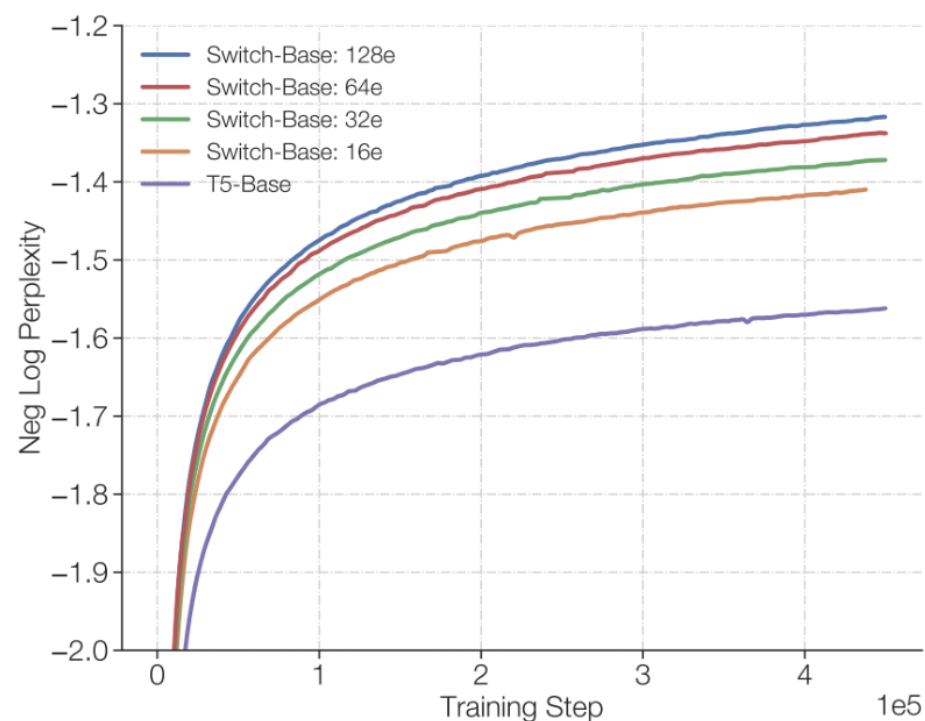
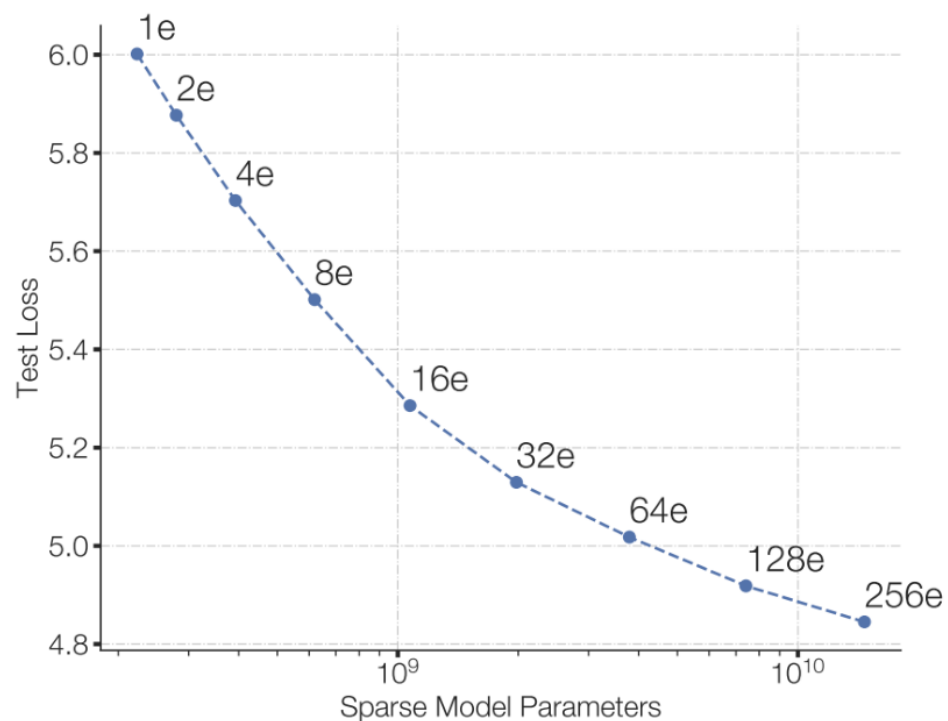
[Fedus et al 2022]

Replace big feedforward with (many) big feedforward networks and a selector layer

You can increase the # experts without affecting FLOPs

Why are MoEs getting popular?

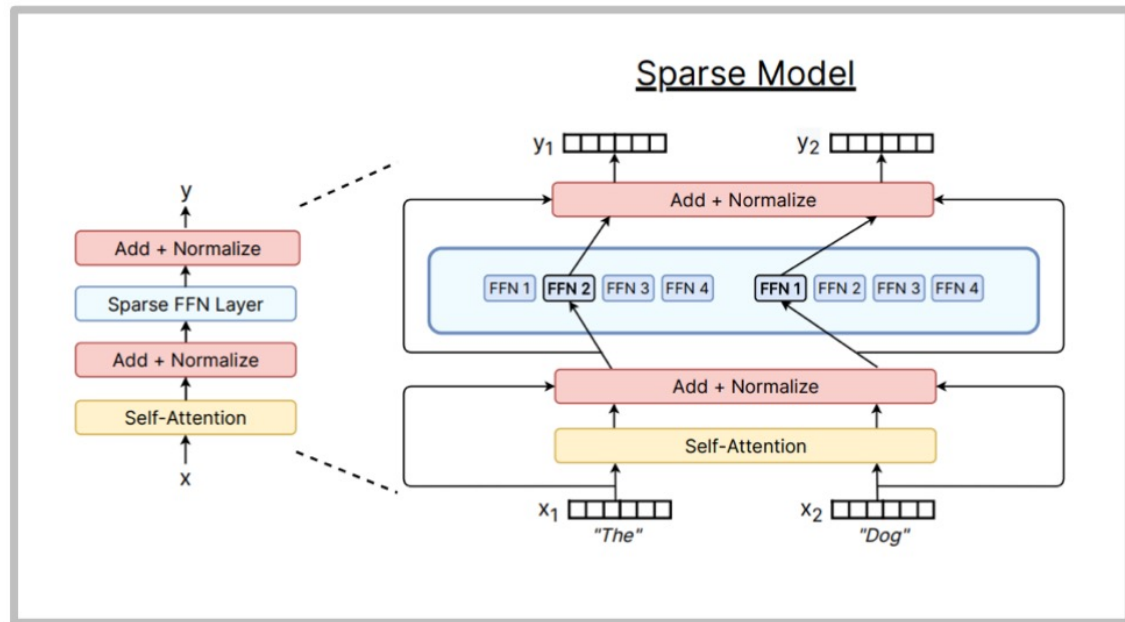
Same FLOP, more param does better



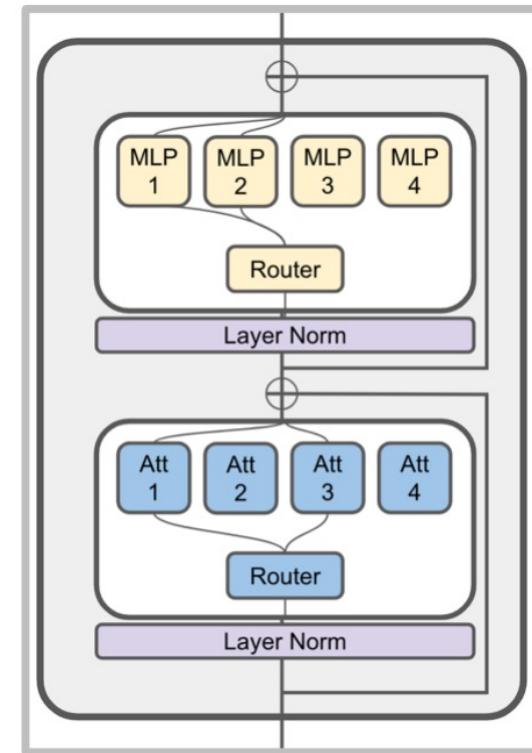
[Fedus et al 2022]

What MoEs generally look like

Typical: replace MLP with MoE layer

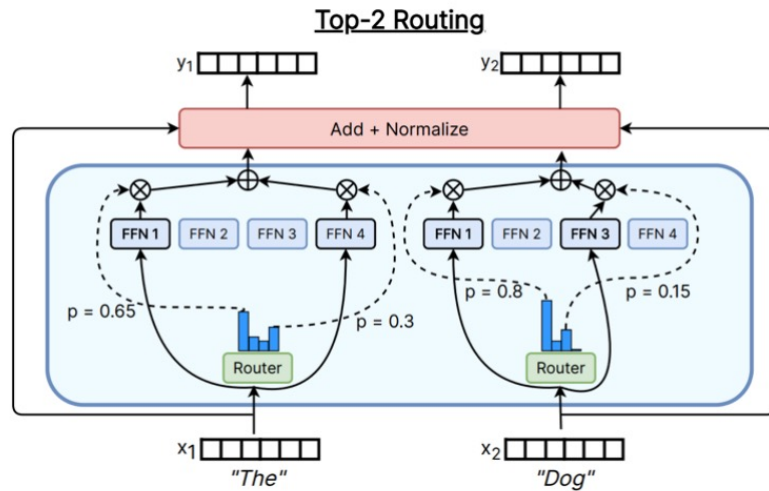


Less common: MoE for attention heads

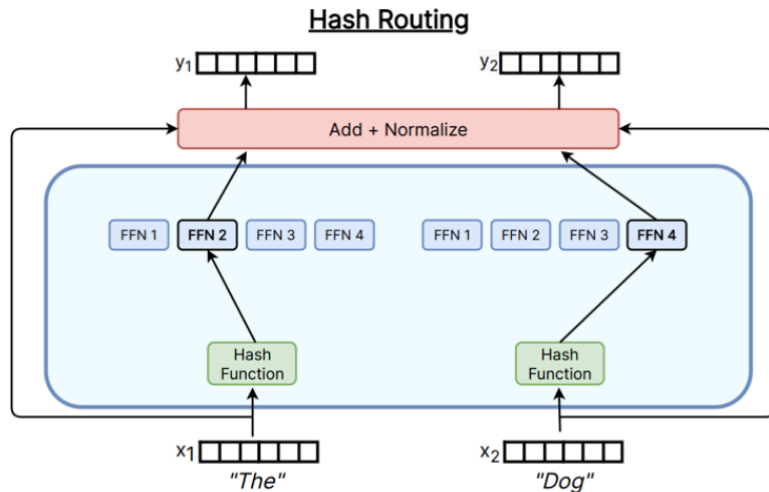


Common routing variants in detail

Top-k



Hashing



Used in most MoEs

Switch Transformer (k=1) Gshard (k=2),
Grok (2), Mixtral (2), Qwen (4), DBRX (4),
DeepSeek (7)

Common baseline

[Fedus et al 2022]

Top-K routing in detail.

Most papers do the old and classic top-k routing. How does this work?

$$\mathbf{h}_t^l = \sum_{i=1}^N \left(\overset{\text{Gating}}{\downarrow} g_{i,t} \text{FFN}_i \left(\mathbf{u}_t^l \right) \right) + \mathbf{u}_t^l,$$
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases}$$
$$s_{i,t} = \text{Softmax}_i \left(\mathbf{u}_t^{lT} \mathbf{e}_i^l \right),$$

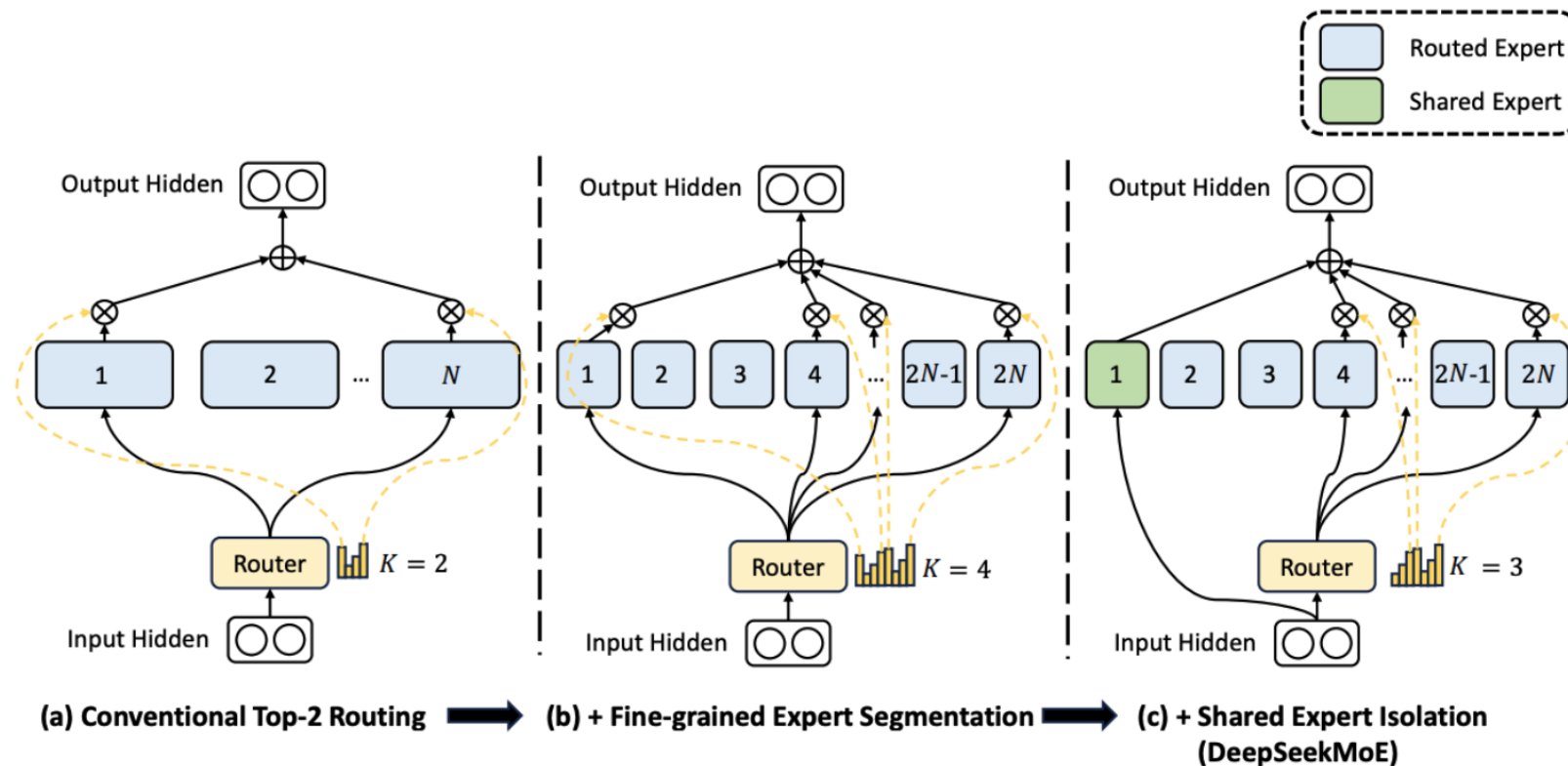
$\uparrow$

Gates selected by a logistic regressor

This is the DeepSeek (V1-2) router (Grok, Qwen do this too)

Mixtral, DBRX,
DeepSeek v3 softmaxes
after the TopK

Recent variations from DeepSeek and other Chinese LMs



Smaller, larger number of experts + a few shared experts that are always on.

(Used in DeepSeek / Qwen, originally from DeepSpeed MoE)

Expert routing setups for recent MoEs

Model	Routed	Active	Shared	Fine-grained ratio
GShard	2048	2	0	
Switch Transformer	64	1	0	
ST-MOE	64	2	0	
Mixtral	8	2	0	
DBRX	16	4	0	
Grok	8	2	0	
DeepSeek v1	64	6	2	1/4
Qwen 1.5	60	4	4	1/8
DeepSeek v3	256	8	1	1/14
OLMoE	64	8	0	1/8
MiniMax	32	2	0	~1/4
Llama 4 (maverick)	128	1	1	1/2

Heuristic balancing losses

Another key issue – systems efficiency requires that we use experts evenly..

For each Switch layer, this auxiliary loss is added to the total model loss during training. Given N experts indexed by $i = 1$ to N and a batch $\mathcal{B}$ with T tokens, the auxiliary loss is computed as the scaled dot-product between vectors f and P ,

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

where f_i is the fraction of tokens dispatched to expert i ,

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax } p(x) = i\} \quad (5)$$

and P_i is the fraction of the router probability allocated for expert i ,²

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x). \quad (6)$$

From the Switch Transformer [Fedus et al 2022]

The derivative with respect to $p_i(x)$ is $\frac{\alpha N}{T^2} \sum \mathbb{1}_{\text{argmax } p(x)=i}$,
so more frequent use = stronger downweighting

DeepSeek v3 variation – per-expert biases

Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

They call this ‘auxiliary loss free balancing’

Complementary Sequence-Wise Auxiliary Loss. Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \quad (17)$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r)), \quad (18)$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \quad (19)$$

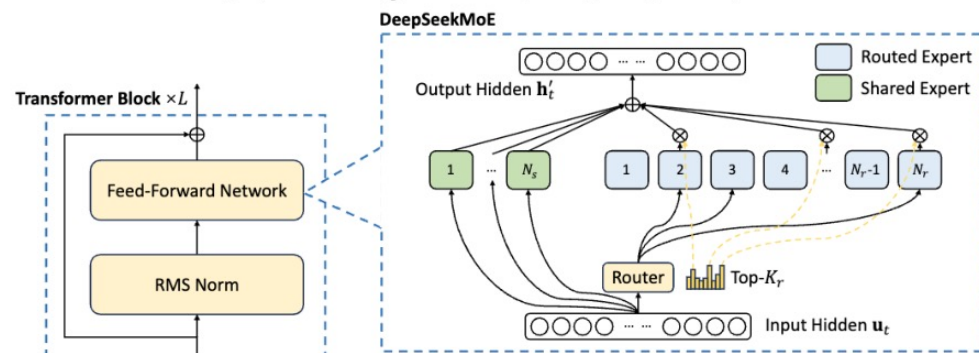
$$P_i = \frac{1}{T} \sum_{t=1}^T s'_{i,t}, \quad (20)$$

(but the approach is not fully aux loss free.)

DeepSeek MoE v3

V2 (671B – 37 active):

Shared (1) + Fine-grained (258) experts, 8 active



New things

Sigmoid+Softmax topK + topM

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t),$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \text{Sigmoid}(\mathbf{u}_t^T \mathbf{e}_i),$$

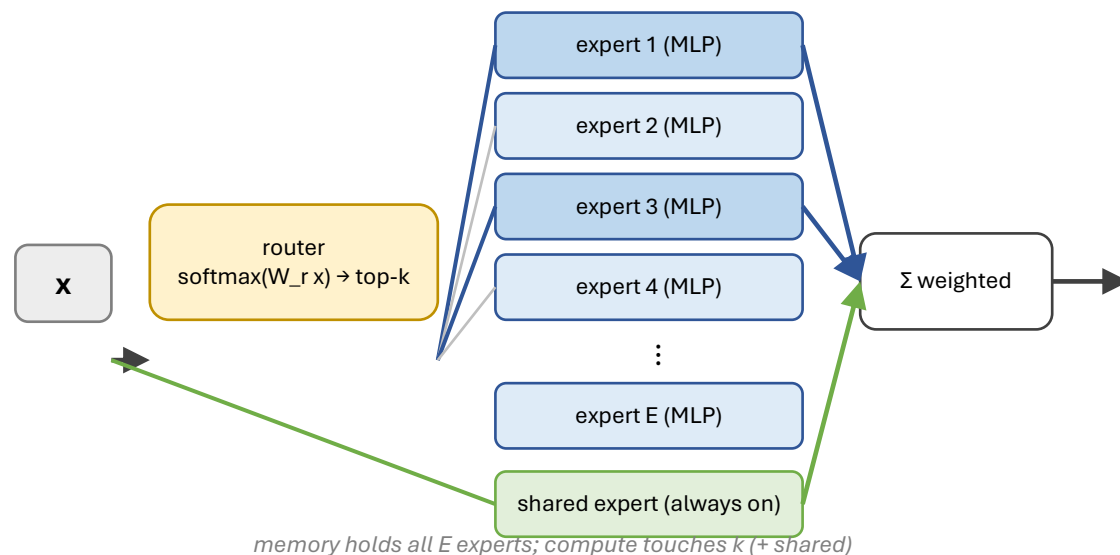
Aux-loss-free + seq-wise aux

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

MoE summary

- ❖ MoEs take advantage of sparsity – not all inputs need the full model
- ❖ Discrete routing is hard, but top-k heuristics seem to work
- ❖ Lots of empirical evidence now that MoEs work, and are cost-effective

Mixture of Experts: more parameters, same FLOPs



Model (date)	Total	Active	Experts
DeepSeek-V3 (Dec 2024)	671B	37B	256 + 1 shared, 8 active
Kimi K2 (Jul 2025)	1T	32B	384 + 1 shared, 8 active
Qwen3-235B-A22B (Apr 2025)	235B	22B	128, 8 active
gpt-oss-120B (Aug 2025)	117B	5.1B	128, 4 active

- Router picks the top-k of E expert MLPs per token; output = weighted sum (+ one always-on shared expert in DeepSeek, Kimi)
- All experts sit in memory, only the active ones are computed → 671B parameters but 37B per token (DeepSeek-V3)
- Load balancing: an auxiliary loss (Switch Transformer, 2021) or DeepSeek's auxiliary-loss-free per-expert bias
- Trend: many small 'fine-grained' experts (128–384) instead of a few large ones (Mixtral 8×7B, 2023: 8 experts, top-2)

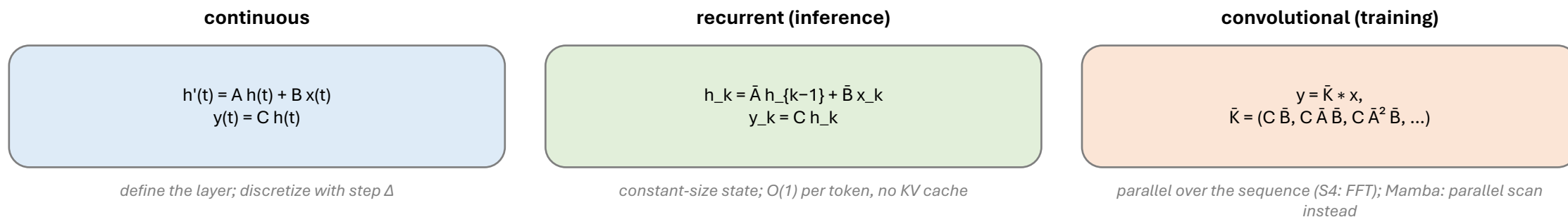
Beyond attention: state-space models and hybrids

H3 → S4/Mamba → Mamba-2 / Gated DeltaNet → production hybrids

H3: Hungry Hungry Hippos — state-space layers as a drop-in for attention

- Before H3, SSM language models (S4) trailed Transformers by several perplexity points; H3 diagnosed two missing skills: **recalling** earlier tokens and **comparing** tokens across the sequence
- H3 stacks two SSMs with multiplicative gating (a shift SSM and a diagonal SSM) so the layer can implement recall and comparison — the same skills attention's induction heads provide
- With just two attention layers added, a hybrid H3 model matched or beat Transformers on OpenWebText perplexity (125M–2.7B); FlashConv (FFT-based) made the SSM convolution fast
- This line led to Mamba (selective SSMs, 2023), Mamba-2 (SSD, 2024) and Gated DeltaNet (2025) — the linear layers used in today's hybrids (next slides)

State-space layers: three views of one model



Mamba (2023): make B , C , Δ functions of the input x_k (selective) — the recurrence can now forget or keep per token, at the cost of the convolution view

- Continuous: $h' = A h + B x$; discretize with a step Δ
- Recurrent (inference): a fixed-size state, $O(1)$ per token, no KV cache
- Convolutional (training): the same map as a long convolution — S4 computes it with FFTs in parallel
- Mamba (Gu & Dao, 2023): B , C , Δ become functions of the input (selective) — the layer can forget or keep per token; training uses a parallel scan instead of the convolution

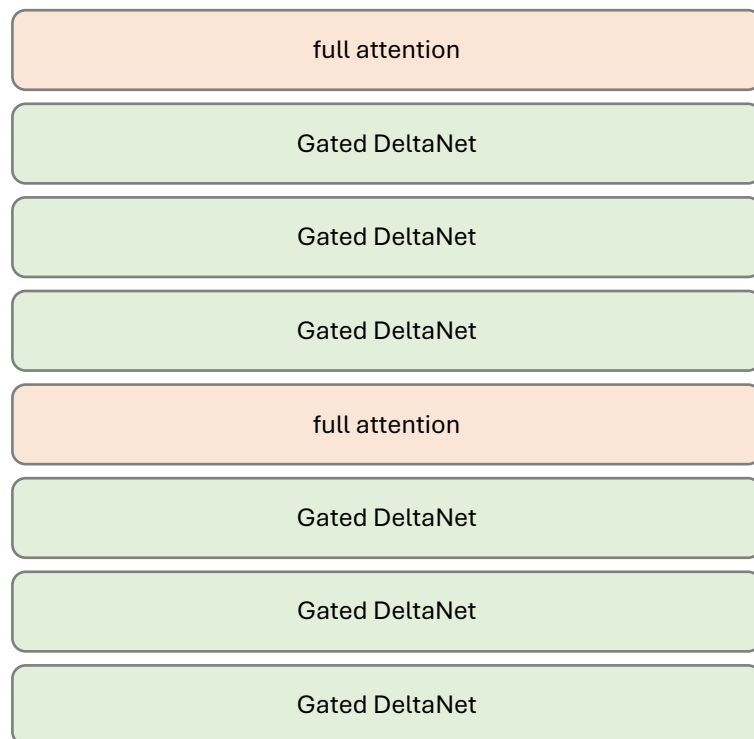
Why SSMs, and why not only SSMs

	training (parallel over T)	inference (per new token)	state / memory	exact recall
recurrent (RNN)	slow (sequential)	fast, $O(1)$	fixed	weak
attention	fast	slow, $O(T)$ + KV cache	grows with T	strong
SSM / linear attention	fast (conv / scan)	fast, $O(1)$	fixed ($d \times d$ per head)	weak $\rightarrow$ hybrids

- Mamba (2023) was the first attention-free LM to match Transformer scaling on language, with linear-time training and constant-memory generation
- Linear attention (drop the softmax) is the same object: a recurrent state $S_t = S_{t-1} + v_t k_t^T$ — Mamba-2's SSD makes the connection exact (Dao & Gu, 2024)
- Weakness: a fixed-size state cannot store every past token — exact recall (needle-in-a-haystack, copying) is worse than attention
- Hence hybrids: keep a few attention layers for recall, use SSM / linear layers for the rest (Jamba 2024 $\rightarrow$ Nemotron-H 2025 $\rightarrow$ Qwen3-Next, Kimi Linear, Nemotron 3)

Hybrids: attention + linear recurrence

3 : 1 pattern (Qwen3-Next / Qwen3.5)



linear: fixed state, $O(T)$; attention: exact recall, KV cache

- Linear-attention / state-space layers (Mamba-2, Gated DeltaNet) keep a constant-size state: no KV growth, $O(T)$ time — but weaker at exact recall, so a few full-attention layers stay
- **Qwen3-Next** (Sep 2025) and **Qwen3.5** (Feb 2026): 3 Gated DeltaNet : 1 gated attention
- **Kimi Linear** (Oct 2025): 3 Kimi Delta Attention : 1 MLA, 1M context; **Nemotron 3** (Dec 2025 – Mar 2026): mostly Mamba-2 with a few GQA layers, 1M context
- **Ling 2.5** (Feb 2026): 7 Lightning Attention : 1 MLA; **xLSTM-7B** (2025): no attention at all
- Gallery trend: dense $\rightarrow$ sparse MoE $\rightarrow$ hybrid (most hybrids are also MoE)

More 2025–26 details worth knowing

- **Multi-token prediction** (DeepSeek-V3, GLM-4.5, Qwen3-Next, MiniMax M2): extra heads predict tokens $t+2$, ... $\rightarrow$ richer training signal and free speculative decoding
- **NoPE layers**: SmolLM3 drops positional encoding in every 4th layer $\rightarrow$ better length generalization; Kimi Linear uses NoPE inside its MLA layers
- **Per-layer embeddings + MatFormer** for elastic small models (Gemma 3n, 2025)
- **Muon optimizer at scale** (Kimi K2, 'MuonClip' = Muon + QK-clip against logit blow-ups) — ties to Lecture 2
- **Engram conditional memory** (DeepSeek, Jan 2026, arXiv 2601.07372): $O(1)$ hashed n-gram lookup tables next to MoE — a second sparsity axis; LongCat-Flash-Lite (Jan 2026) ships n-gram embeddings
- **mHC**, manifold-constrained hyper-connections (DeepSeek, Dec 2025): widen the residual stream while restoring the identity-mapping property

Where to look: the LLM Architecture Gallery

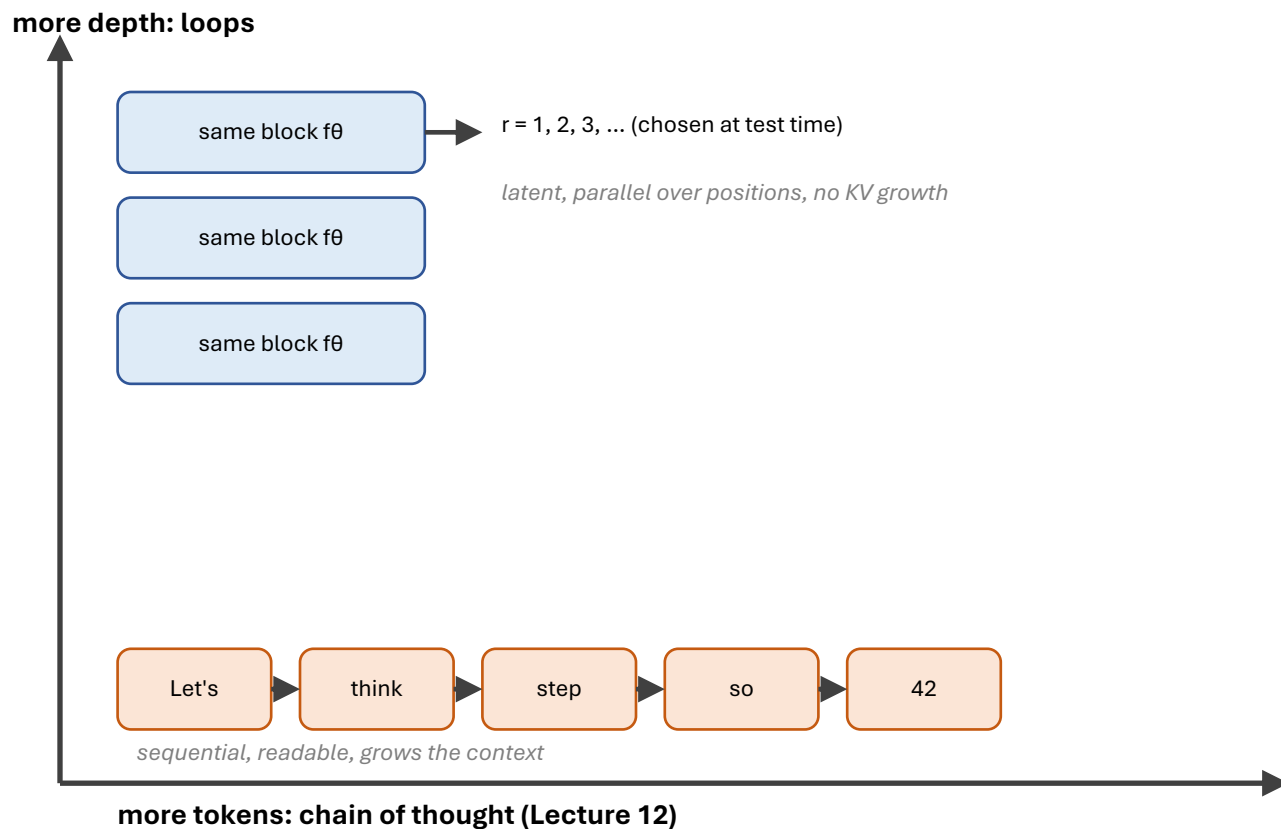


- 102 models (Sep 2026) with architecture diagrams and fact sheets: params, active params, context, attention type, KV-cache growth
- A side-by-side architecture diff tool; one 'looped' entry (Ouro) since Oct 2025
- sebastianraschka.com/llm-architecture-gallery — companion to 'The Big LLM Architecture Comparison' (2025)

Looped transformers: depth as test-time compute

Part II — re-using one block many times

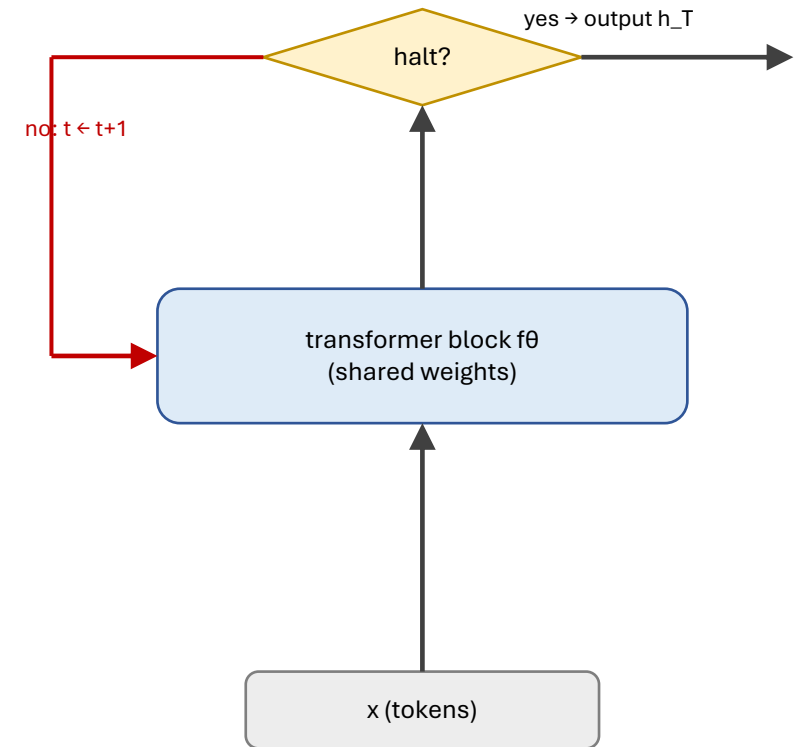
Two ways to spend more compute at inference



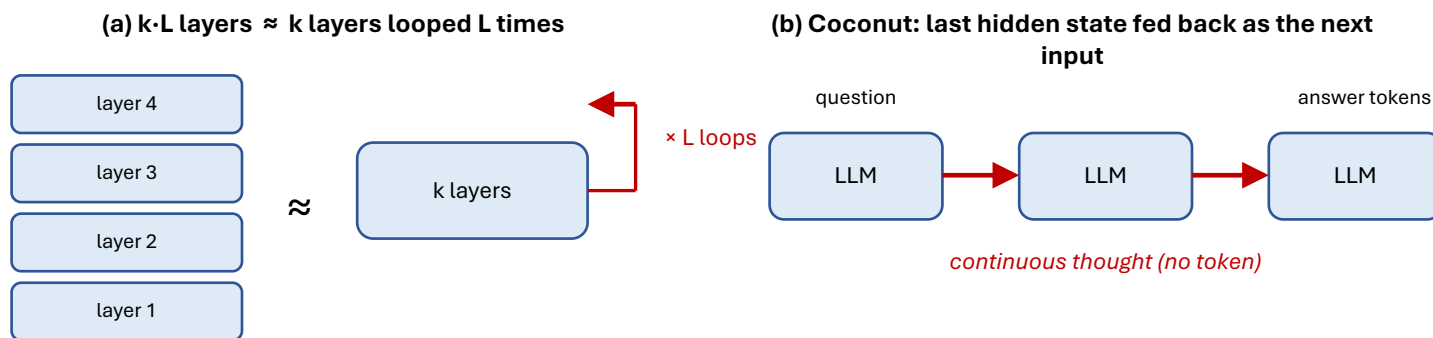
- **Chain of thought** (Lecture 12): sequential and visible; every step is a token, so context and KV cache grow
- **Looping**: re-apply the same block to the hidden state — latent, parallel over all positions, no extra tokens
- Recurrent depth = weight sharing across 'virtual' layers: the parameter count is fixed, the depth is chosen at test time
- The two can be combined: think in loops, then emit a chain of thought (Ouro-Thinking, LOTUS — slides 16 and 18)

The idea is old

- **Universal Transformer** (Dehghani et al., 2018): one block applied T times; **Adaptive Computation Time** (Graves, 2016) halts per position
- **ALBERT** (Lan et al., 2019): cross-layer parameter sharing for a compact BERT
- **Looped Transformers as Programmable Computers** (Giannou et al., ICML 2023): a 13-layer looped transformer emulates a small instruction-set computer — calculator, linear algebra, in-context learning by backprop
- **Looped transformers are better at learning learning algorithms** (Yang et al., ICLR 2024): matches a standard transformer on in-context data fitting with $< 10\%$ of the parameters
- **Deep equilibrium models** (Bai et al., 2019): the infinite-loop limit, solved as a fixed point

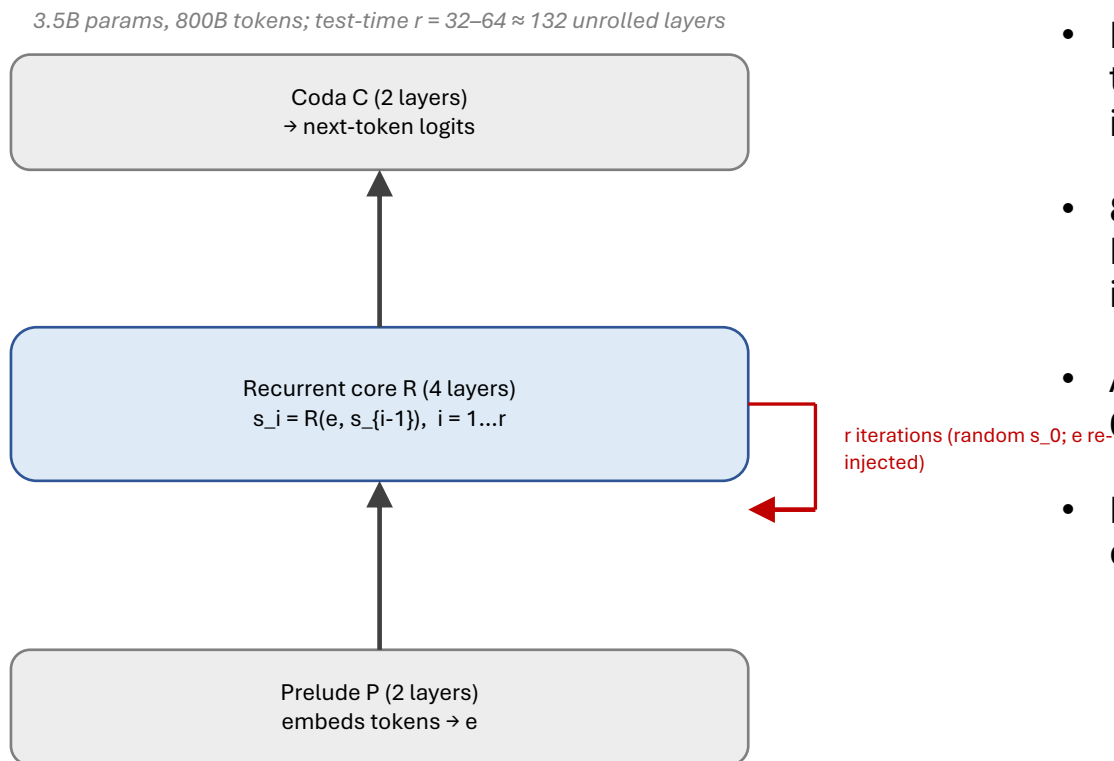


Theory: loops behave like latent reasoning steps



- **Saunshi et al.** (ICLR 2025): a k -layer block looped L times nearly matches a $k \cdot L$ -layer model on reasoning tasks (addition, p-hop, math word problems) with a fraction of the parameters
- Loops act like CoT steps in latent space: L loops $\approx L$ latent thoughts; iterative algorithms are the theoretical reason
- Perplexity favors parameters; reasoning favors depth — the paper also derives a looping-inspired regularizer for standard models
- **Coconut** (Hao et al., Dec 2024): feed the last hidden state back as the next input embedding — a 'continuous thought' instead of a word; can keep several alternatives alive at once (BFS-like)

Scaling recurrent depth: Huginn-3.5B

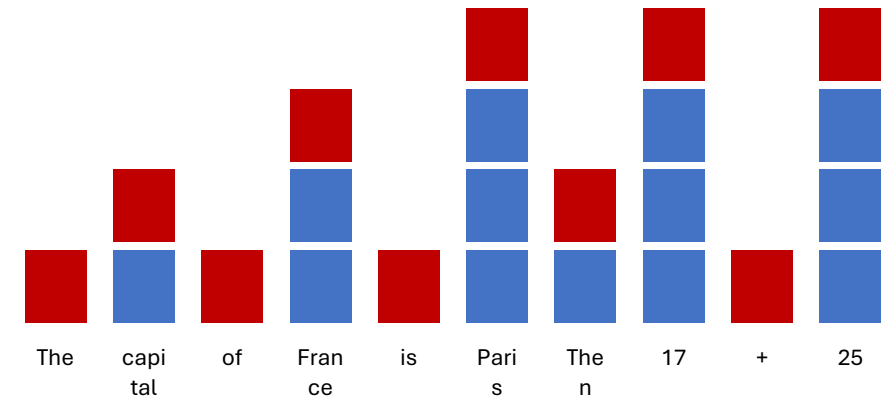


- Prelude (2 layers) → recurrent core (4 layers, iterated r times; the embedded input e is re-injected every iteration; random initial state) → coda (2 layers)
- 800B training tokens; r sampled per step (log-normal Poisson, mean 32); backprop truncated to the last 8 iterations
- At test time you pick r : accuracy keeps improving up to ~32–64 iterations ($r = 32 \approx$ an unrolled 132-layer model)
- No CoT data needed; enables zero-shot per-token adaptive exit and KV-cache sharing across iterations

Ouro and Mixture-of-Recursions (2025)

- **Ouro** (Zhu et al., Oct 2025): 1.4B and 2.6B looped LMs pretrained on 7.7T tokens with 4 recurrent steps and an entropy-regularized learned exit gate
- Authors report parity with standard models of up to 12B parameters on reasoning benchmarks and attribute the gain to better knowledge manipulation, not more stored knowledge
- First looped model in the architecture gallery (48 shared layers × 4 passes)
- **Mixture-of-Recursions** (Bae et al., NeurIPS 2025): a lightweight router chooses how many recursions each token gets; recursion-wise KV caching → adaptive depth at equal FLOPs (135M–1.7B models)

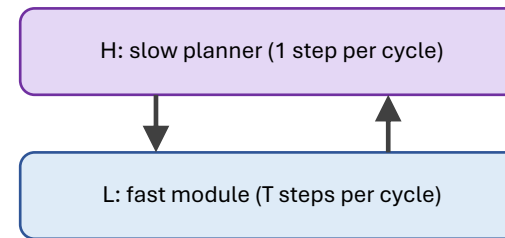
recursions per token (red = exit); MoR routes each token to its own depth; Ouro uses a learned exit gate



Tiny recursive reasoners

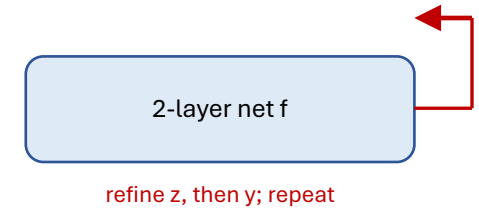
- **HRM** (Wang et al., Jun 2025): 27M params; two recurrent modules at different timescales (slow planner H, fast worker L); ~1000 training examples per task
- **TRM** (Jolicoeur-Martineau, Oct 2025): a single 2-layer network recursed; 7M params; reports 45% on ARC-AGI-1 and 8% on ARC-AGI-2 (public eval) — above much larger LLMs on these puzzles
- Caveat: trained per task family (Sudoku, mazes, ARC) with heavy augmentation — not a general language model
- Lesson: for algorithmic puzzles, depth by recursion beats parameters

(a) HRM: two modules, two timescales



27M params · ~1000 examples per task

(b) TRM: one 2-layer net, recursed



7M params · reports 45% ARC-AGI-1, 8% ARC-AGI-2
(public eval)

2026: what we know now

Apr 2026
arXiv 2604.07822

Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers — Kohli, Parthasarathy, Sun, Yao

Recurrent-depth transformers compose facts that were never paired in training (via a three-stage grokking process) and extrapolate from shallow to deeper multi-hop reasoning by adding inference-time recurrence — but too many iterations 'overthink' and degrade predictions.

May 2026
arXiv 2605.23872

Training-Free Looped Transformers — Chen, Li, Liang, Lao, Liu

A test-time wrapper loops a contiguous mid-stack block of a frozen checkpoint; naive re-application usually hurts, but treating the loop as smaller damped ODE sub-steps (a pre-norm block $\approx$ one forward-Euler step) gains e.g. +2.64 pp MMLU-Pro on Qwen3-4B-Instruct, across seven dense / MoE / MLA+MoE families.

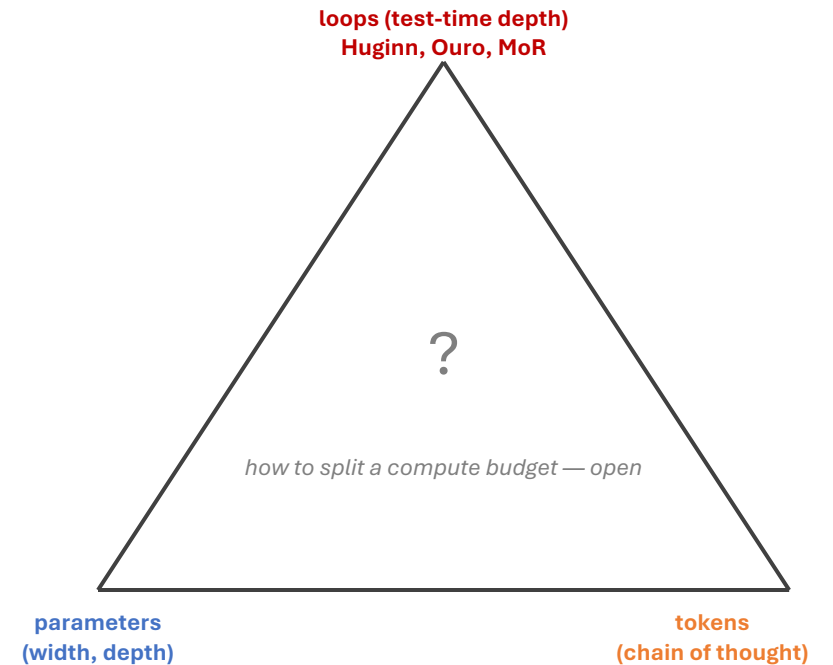
Jun 2026
arXiv 2606.31779

Bridging the Gap Between Latent and Explicit Reasoning with Looped Transformers — Fan, Svete, Lee

Latent CoT lagged explicit CoT beyond 1B parameters; LOTUS — a looped padded transformer that processes K latent blocks in parallel for R iterations, with a loss on each latent position's gold CoT-step token — closes the gap at 3B while cutting thought-phase latency 2.5–6.9 $\times$.

Open questions

- Three scaling axes for reasoning — parameters, loops (depth), tokens (CoT): how to split a compute budget between them is open
- Latent reasoning is not human-readable: efficiency vs. monitorability (a chain of thought can be read, audited and rewarded — Lecture 12)
- Memory: loops share weights, but each iteration can add KV; Huginn and MoR share KV across iterations — the right cache policy is open
- Relatives: diffusion LMs (iterative refinement of the whole sequence), test-time training (update weights per input), deep equilibrium models
- Will frontier labs adopt loops at scale? So far: $\leq 3.5\text{B}$ trained from scratch (Huginn, Ouro) and training-free retrofits (2026)



Summary

- The 2026 block: RMSNorm pre-norm, RoPE, GQA or MLA, SwiGLU, QK-norm; no dropout, no biases, no position table
- Inference is bound by the KV cache → GQA → MLA → sliding-window / sparse attention → hybrids with linear recurrence
- MoE decouples parameters from FLOPs (DeepSeek-V3: 671B total, 37B active) and is now the default for large open models
- Looped transformers reuse one block r times: depth as test-time compute, latent CoT, far fewer parameters (Universal Transformer 2018 → Huginn, Ouro, MoR 2025)
- 2026 evidence: loops compose facts and extrapolate depth but can overthink; training-free loops help frozen checkpoints; LOTUS matches explicit CoT at 3B